

## NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

[illegible]

Vĩnh Long, ngày ..... tháng ...8... năm ...2026...

**Giảng viên hướng dẫn**  
(Ký tên và ghi rõ họ tên)

[illegible]

**Thành viên hội đồng**  
(Ký tên và ghi rõ họ tên)

---

## LỜI CẢM ƠN

Đồ án kết thúc môn Công nghệ phần mềm “Xây dựng ứng dụng quản lý thư viện” được hoàn thành nhờ vào sự giúp đỡ, hỗ trợ từ rất nhiều tập thể và cá nhân.

Trước hết, em xin chân thành cảm ơn tất cả các giảng viên Khoa Công nghệ thông tin – Trường Kỹ thuật & Công nghệ - Đại học Trà Vinh đã nhiệt tình giảng dạy, và đã trang bị cho em những kiến thức từ cơ bản đến nâng cao, tạo điều kiện thuận lợi cho em trong quá trình học tập, nghiên cứu, hoàn thành đề tài.

Đặc biệt, em xin bày tỏ lòng biết ơn sâu sắc cũng như gửi lời cảm ơn chân thành đến TS. Nguyễn Bảo Ân đã dành thời gian chỉ bảo tận tình cũng như truyền đạt cho em những kiến thức quý báu trong suốt thời gian em thực hiện đề tài đồ án cơ sở ngành.

Với vốn kiến thức và sự hiểu biết còn nhiều hạn hẹp nên bài báo cáo của em sẽ không tránh khỏi những thiếu sót. Em mong nhận được ý kiến đóng góp từ quý Thầy/Cô trong hội đồng đánh giá đồ án cơ sở ngành để em có thể đúc kết được nhiều bài học và kinh nghiệm cho bản thân, từ đó giúp cho em phát triển hơn trên con đường học tập và làm việc sau này.

Cuối cùng, em xin gửi đến quý Thầy/Cô lời chúc sức khỏe và thành công trong sự nghiệp giảng dạy! Em xin chân thành cảm ơn!

---

## MỤC LỤC

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| MỞ ĐẦU.....                                                           | 10 |
| CHƯƠNG 1: TỔNG QUAN.....                                              | 21 |
| CHƯƠNG 2: NGHIÊN CỨU LÝ THUYẾT .....                                  | 24 |
| 2.1    Tổng quan về Công nghệ phần mềm.....                           | 24 |
| 2.1.1    Khái niệm.....                                               | 24 |
| 2.1.2    Quy trình phát triển phần mềm .....                          | 24 |
| 2.1.3    Các mô hình phát triển phần mềm .....                        | 24 |
| 2.2    Hệ thống quản lý thư viện .....                                | 31 |
| 2.2.1    Giới thiệu về Hệ thống quản lý thư viện.....                 | 31 |
| 2.2.2    Mục tiêu của hệ thống.....                                   | 31 |
| 2.2.3    Chức năng của hệ thống.....                                  | 31 |
| 2.2.4    Quản lý và bảo mật .....                                     | 32 |
| 2.2.5    Mô tả chi tiết hoạt động và quan hệ giữa các đối tượng ..... | 32 |
| 2.3    Tổng quan về NodeJS.....                                       | 33 |
| 2.3.1    NodeJS là gì? .....                                          | 33 |
| 2.3.2    Các đặc tính của NodeJS.....                                 | 34 |
| 2.4    Tổng quan về React.....                                        | 34 |
| 2.4.1    React là gì?.....                                            | 34 |
| CHƯƠNG 3: HIỆN THỰC HÓA NGHIÊN CỨU .....                              | 41 |
| 3.1    Quy trình nghiên cứu.....                                      | 41 |
| 3.2    Phân tích yêu cầu hệ thống.....                                | 41 |
| 3.2.1    Yêu cầu chức năng.....                                       | 41 |
| 3.2.2    Yêu cầu phi chức năng.....                                   | 41 |
| 3.3    Cấu trúc use case của hệ thống.....                            | 42 |
| 3.4    Các chức năng chính của hệ thống.....                          | 42 |
| 3.5    Công nghệ sử dụng .....                                        | 45 |
| CHƯƠNG 4: KẾT QUẢ NGHIÊN CỨU.....                                     | 46 |
| 4.1    Màn hình tổng quan hệ thống thư viện .....                     | 46 |
| 4.2    Quản lý sách .....                                             | 47 |
| 4.3    Kết quả xây dựng mô hình .....                                 | 47 |

---

|                                             |    |
|---------------------------------------------|----|
| CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN..... | 49 |
| 5.1    Kết luận .....                       | 49 |
| 5.2    Hướng phát triển.....                | 50 |
| DANH MỤC TÀI LIỆU THAM KHẢO .....           | 51 |

---

## DANH MỤC HÌNH ẢNH

|                                                    |    |
|----------------------------------------------------|----|
| Hình 2.1 Mô hình thác nước – Waterfall model ..... | 25 |
| Hình 2.2 Mô hình V-V-Shaped Model .....            | 26 |
| Hình 4.1 Dashboard của hệ thống .....              | 46 |
| Hình 4.2 Màn hình quản lý sách .....               | 47 |
| Hình 4.3 Màn hình quản lý phiếu mượn .....         | 48 |
| Hình 4.4 Màn hình quản lý người dùng .....         | 48 |

---

## DANH MỤC BẢNG BIỂU

|                                                       |    |
|-------------------------------------------------------|----|
| Bảng 1. Yêu cầu chức năng của hệ thống .....          | 41 |
| Bảng 2. Danh sách các use case của hệ thống .....     | 42 |
| Bảng 3. Tổng quan các chức năng của hệ thống.....     | 44 |
| Bảng 4. Các công nghệ được sử dụng trong đề tài ..... | 45 |

---

## TÓM TẮT ĐỒ ÁN

Trong bối cảnh công nghệ thông tin ngày càng được ứng dụng rộng rãi trong hoạt động quản lý, việc xây dựng các hệ thống phần mềm nhằm tự động hóa và nâng cao hiệu quả quản lý thư viện là một nhu cầu thực tế. Phương pháp quản lý thủ công có thể gây khó khăn trong việc lưu trữ, tìm kiếm và cập nhật thông tin sách, quản lý người dùng cũng như theo dõi quá trình mượn và trả sách. Vì vậy, đề tài “Xây dựng ứng dụng quản lý thư viện” được thực hiện nhằm xây dựng một hệ thống quản lý thư viện trên nền tảng web, hỗ trợ các nghiệp vụ cơ bản và giảm bớt sự phụ thuộc vào quá trình xử lý thủ công.

Hệ thống được xây dựng với mục tiêu quản lý tập trung thông tin sách, tài khoản người dùng và các hoạt động mượn – trả sách. Hệ thống cung cấp cơ chế xác thực và phân quyền với ba nhóm người dùng chính gồm Admin, Librarian và Member. Mỗi nhóm người dùng được cấp các quyền phù hợp với vai trò của mình. Thành viên có thể đăng nhập, xem thông tin sách, thực hiện mượn sách và theo dõi lịch sử mượn trả. Thủ thư có thể quản lý thông tin sách và thực hiện các nghiệp vụ liên quan đến mượn – trả. Quản trị viên có quyền quản lý và kiểm soát các chức năng quan trọng của hệ thống.

Các chức năng chính của hệ thống bao gồm đăng ký và đăng nhập tài khoản, xác thực người dùng, phân quyền truy cập, quản lý thông tin sách, xem chi tiết sách, quản lý số lượng sách, tạo phiếu mượn, theo dõi tình trạng mượn, trả sách và xử lý trường hợp trả sách quá hạn. Khi người dùng trả sách, hệ thống cập nhật trạng thái phiếu mượn và số lượng sách có sẵn. Đối với trường hợp trả sách quá hạn, hệ thống xác định thời gian quá hạn và tính khoản tiền phạt tương ứng theo quy định được thiết lập.

Về mặt công nghệ, hệ thống được phát triển theo mô hình ứng dụng web với sự phân tách giữa Frontend, Backend và cơ sở dữ liệu. React kết hợp với Vite được sử dụng để xây dựng giao diện người dùng; Node.js và Express.js được sử dụng để xây dựng Backend và cung cấp các RESTful API; MongoDB kết hợp với Mongoose được sử dụng để lưu trữ và quản lý dữ liệu. Bên cạnh đó, hệ thống sử dụng các cơ

chế xác thực, mã hóa mật khẩu và phân quyền nhằm tăng tính bảo mật. Các API được kiểm thử bằng Postman trong quá trình phát triển.

Kết quả của đề tài là xây dựng được một ứng dụng quản lý thư viện với các chức năng cốt lõi phục vụ việc quản lý sách, người dùng và hoạt động mượn – trả. Các chức năng chính đã được triển khai và kiểm thử trong môi trường phát triển. Qua quá trình thực hiện, đề tài giúp nhóm vận dụng kiến thức về phân tích yêu cầu, thiết kế hệ thống, cơ sở dữ liệu, lập trình Web, xác thực, phân quyền và kiểm thử phần mềm vào một bài toán thực tế. Hệ thống đồng thời tạo nền tảng để có thể tiếp tục phát triển các chức năng nâng cao như thông báo tự động, đặt trước sách, thanh toán trực tuyến, quản lý mã vạch hoặc QR Code và thống kê hoạt động thư viện trong tương lai.

## MỞ ĐẦU

### 1. Lý do chọn đề tài

Trong bối cảnh công nghệ thông tin ngày càng phát triển và được ứng dụng rộng rãi trong nhiều lĩnh vực của đời sống, việc sử dụng các hệ thống phần mềm để hỗ trợ quản lý và tự động hóa các công việc thủ công đang trở thành một nhu cầu thiết yếu. Trong lĩnh vực giáo dục, thư viện đóng vai trò quan trọng trong việc cung cấp tài liệu phục vụ học tập, giảng dạy và nghiên cứu. Tuy nhiên, tại một số thư viện trường học hoặc thư viện có quy mô vừa và nhỏ, việc quản lý sách, độc giả và hoạt động mượn trả vẫn còn phụ thuộc nhiều vào phương pháp thủ công hoặc các công cụ quản lý đơn giản. Điều này có thể gây khó khăn trong việc cập nhật thông tin, kiểm soát số lượng sách và theo dõi tình trạng mượn trả của người dùng.

Trong thực tế, một thư viện cần quản lý một lượng lớn thông tin như thông tin sách, tác giả, thể loại, số lượng sách, người dùng, lịch sử mượn trả và tình trạng sách. Nếu các thông tin này được quản lý bằng sổ sách hoặc các bảng dữ liệu riêng lẻ, nhân viên thư viện có thể mất nhiều thời gian để tìm kiếm và cập nhật dữ liệu. Khi số lượng sách và người sử dụng tăng lên, nguy cơ xảy ra sai sót trong quá trình nhập liệu, thống kê số lượng sách hoặc theo dõi các khoản phạt cũng tăng theo. Bên cạnh đó, việc kiểm tra một người dùng đang mượn những cuốn sách nào, sách nào đã quá hạn hoặc số lượng sách còn có thể cho mượn cũng trở nên khó khăn nếu không có một hệ thống quản lý tập trung.

Đối với người sử dụng thư viện, phương thức quản lý thủ công cũng có thể làm giảm sự thuận tiện trong quá trình sử dụng dịch vụ. Người dùng có thể gặp khó khăn khi tìm kiếm thông tin về sách, kiểm tra tình trạng sách hoặc theo dõi lịch sử mượn trả của bản thân. Nhân viên thư viện cũng phải thực hiện nhiều công việc lặp lại như kiểm tra thông tin người dùng, kiểm tra số lượng sách còn lại, cập nhật trạng thái sách sau khi mượn hoặc trả và xác định sách trả quá hạn. Vì vậy, việc xây dựng một ứng dụng quản lý thư viện có khả năng tập trung dữ liệu và hỗ trợ tự động hóa các nghiệp vụ cơ bản là cần thiết và có tính thực tiễn cao.

Xuất phát từ những vấn đề trên, nhóm lựa chọn đề tài **“Xây dựng ứng dụng quản lý thư viện”** nhằm xây dựng một hệ thống có khả năng hỗ trợ các nghiệp vụ quản lý thư viện một cách tập trung, chính xác và thuận tiện hơn. Hệ thống hướng đến việc quản lý thông tin sách, tài khoản người dùng, hoạt động mượn và trả sách, đồng thời phân quyền sử dụng dựa trên vai trò của từng đối tượng. Trong đó, các vai trò như quản trị viên, thủ thư và thành viên sẽ có những quyền hạn khác nhau nhằm đảm bảo mỗi người dùng chỉ được thực hiện các chức năng phù hợp với trách nhiệm của mình.

Một trong những vấn đề thực tế được đề tài quan tâm là quá trình quản lý mượn và trả sách. Hệ thống cần có khả năng ghi nhận thời điểm mượn, thời hạn trả, trạng thái của phiếu mượn và thời điểm trả thực tế. Khi người dùng trả sách quá hạn, hệ thống có thể xác định số ngày quá hạn và hỗ trợ tính toán tiền phạt theo quy định được thiết lập. Việc này giúp giảm sự phụ thuộc vào việc tính toán thủ công và hạn chế sai sót trong quá trình quản lý. Đồng thời, thông tin về số lượng sách có sẵn cũng được cập nhật theo hoạt động mượn và trả, giúp thủ thư dễ dàng kiểm soát tình trạng tài nguyên của thư viện.

Bên cạnh ý nghĩa thực tiễn, đề tài còn được lựa chọn vì phù hợp với nội dung của môn học Công nghệ phần mềm. Quá trình xây dựng hệ thống giúp nhóm có cơ hội vận dụng các kiến thức đã học vào một bài toán thực tế, từ bước khảo sát và phân tích yêu cầu, xác định tác nhân và chức năng của hệ thống, thiết kế cơ sở dữ liệu, thiết kế kiến trúc phần mềm cho đến lập trình, kiểm thử và hoàn thiện sản phẩm. Thông qua quá trình này, nhóm có thể hiểu rõ hơn về quy trình phát triển một sản phẩm phần mềm thay vì chỉ tiếp cận các kiến thức một cách lý thuyết.

Ngoài ra, việc lựa chọn đề tài còn giúp nhóm thực hành các công nghệ phát triển ứng dụng web hiện đại. Hệ thống được xây dựng theo mô hình tách biệt giữa giao diện người dùng và máy chủ, trong đó React được sử dụng để xây dựng giao diện phía người dùng, Node.js kết hợp với Express.js được sử dụng để xây dựng phía máy chủ và MongoDB được sử dụng để lưu trữ dữ liệu. Việc kết hợp các công nghệ này giúp nhóm có thêm kinh nghiệm trong việc xây dựng

API, kết nối cơ sở dữ liệu, xác thực người dùng, phân quyền truy cập và kết nối giữa Frontend với Backend.

Từ những lý do trên, đề tài “**Xây dựng ứng dụng quản lý thư viện**” được lựa chọn với mong muốn tạo ra một sản phẩm có khả năng giải quyết những nghiệp vụ cơ bản trong hoạt động quản lý thư viện, đồng thời giúp nhóm vận dụng tổng hợp kiến thức của môn Công nghệ phần mềm vào một dự án thực tế. Thông qua đề tài, nhóm không chỉ hướng đến việc hoàn thành một sản phẩm phần mềm đáp ứng các yêu cầu đặt ra mà còn tích lũy kinh nghiệm trong quá trình phân tích, thiết kế, phát triển, kiểm thử và quản lý một hệ thống phần mềm hoàn chỉnh.

## **2. Mục đích nghiên cứu**

Mục đích của đề tài “Xây dựng ứng dụng quản lý thư viện” là phân tích, thiết kế và xây dựng một hệ thống phần mềm hỗ trợ các nghiệp vụ quản lý thư viện, giúp việc quản lý sách, người dùng và hoạt động mượn trả được thực hiện một cách tập trung, chính xác, thuận tiện và hiệu quả hơn so với phương pháp quản lý thủ công.

Trước hết, đề tài hướng đến việc xây dựng một hệ thống có khả năng quản lý thông tin sách một cách đầy đủ và có hệ thống. Các thông tin như tên sách, tác giả, mã ISBN, thể loại, nhà xuất bản, năm xuất bản, số lượng và trạng thái hoạt động của sách được lưu trữ tập trung trong cơ sở dữ liệu. Người có quyền quản lý có thể thực hiện các thao tác thêm, cập nhật, xem và quản lý thông tin sách. Việc này giúp hạn chế tình trạng thất lạc thông tin, trùng lặp dữ liệu và giảm thời gian tìm kiếm hoặc cập nhật thông tin.

Bên cạnh quản lý sách, đề tài tập trung xây dựng chức năng quản lý tài khoản và phân quyền người dùng. Hệ thống phân chia người dùng thành các vai trò như Admin, Librarian và Member, trong đó mỗi vai trò được cấp các quyền phù hợp với nhiệm vụ của mình. Cơ chế này giúp tăng tính bảo mật cho hệ thống, hạn chế việc người dùng truy cập hoặc thực hiện những chức năng không thuộc quyền hạn của mình. Đồng thời, việc xác thực tài khoản giúp hệ thống xác

định chính xác người đang sử dụng và kiểm soát các thao tác được thực hiện trên hệ thống.

Một mục đích quan trọng khác của đề tài là hỗ trợ quản lý quy trình mượn và trả sách. Hệ thống cho phép ghi nhận thông tin người mượn, sách được mượn, ngày mượn, hạn trả và trạng thái của phiếu mượn. Khi người dùng thực hiện trả sách, hệ thống cập nhật trạng thái phiếu mượn và số lượng sách có sẵn. Trong trường hợp người dùng trả sách quá hạn, hệ thống có thể xác định thời gian quá hạn và tính tiền phạt theo quy định. Qua đó, các nghiệp vụ thường xuyên của thư viện được xử lý nhanh chóng và giảm sự phụ thuộc vào việc tính toán, ghi chép thủ công.

Đề tài cũng hướng đến việc xây dựng giao diện web trực quan và dễ sử dụng, giúp các nhóm người dùng khác nhau có thể tiếp cận hệ thống một cách thuận tiện. Thành viên có thể đăng nhập, xem thông tin sách, thực hiện mượn sách và theo dõi lịch sử mượn trả của bản thân. Thủ thư có thể quản lý sách và các hoạt động mượn trả, trong khi quản trị viên có quyền quản lý và kiểm soát hệ thống ở mức cao hơn. Việc thiết kế giao diện theo từng vai trò giúp người dùng tập trung vào những chức năng cần thiết và hạn chế sự phức tạp khi sử dụng.

Về mặt kỹ thuật, đề tài nhằm giúp nhóm vận dụng các kiến thức của môn Công nghệ phần mềm vào quá trình phát triển một sản phẩm thực tế. Nhóm thực hiện các bước từ khảo sát bài toán, phân tích yêu cầu, thiết kế hệ thống, thiết kế cơ sở dữ liệu, xây dựng Backend và Frontend, tích hợp các thành phần, kiểm thử và hoàn thiện sản phẩm. Thông qua quá trình này, nhóm có thể hiểu rõ hơn về quy trình phát triển phần mềm và rèn luyện khả năng làm việc với một hệ thống có nhiều thành phần liên kết với nhau.

Ngoài ra, đề tài còn nhằm giúp nhóm có kinh nghiệm thực tế trong việc sử dụng các công nghệ như React, Vite, Node.js, Express.js, MongoDB và Mongoose, đồng thời thực hành xây dựng RESTful API, xác thực người dùng, phân quyền truy cập và kết nối giữa giao diện với máy chủ. Việc kiểm thử các chức năng thông qua Postman cũng giúp nhóm đánh giá tính đúng đắn của API trước khi tích hợp vào giao diện người dùng.

Tóm lại, mục đích nghiên cứu của đề tài không chỉ là tạo ra một ứng dụng có khả năng hỗ trợ các nghiệp vụ cơ bản của thư viện mà còn giúp nhóm áp dụng một cách tổng hợp các kiến thức về phân tích yêu cầu, thiết kế, lập trình, cơ sở dữ liệu, kiểm thử và quản lý phần mềm. Qua đó, đề tài hướng đến việc tạo ra một sản phẩm có tính thực tiễn, đáp ứng được các yêu cầu cơ bản của một hệ thống quản lý thư viện và tạo nền tảng để tiếp tục mở rộng thêm các chức năng trong tương lai.

### **3. Đối tượng nghiên cứu**

Đối tượng nghiên cứu của đề tài bao gồm: Đối tượng nghiên cứu của đề tài “Xây dựng ứng dụng quản lý thư viện” là quy trình quản lý và các nghiệp vụ cơ bản trong hoạt động của một thư viện, đồng thời nghiên cứu cách ứng dụng công nghệ phần mềm để xây dựng hệ thống hỗ trợ các nghiệp vụ này trên nền tảng web.

Cụ thể, đề tài tập trung nghiên cứu các đối tượng và thành phần chính sau:

- Thứ nhất, nghiên cứu đối tượng người dùng của hệ thống. Hệ thống bao gồm ba nhóm người dùng chính là Admin, Librarian và Member. Mỗi nhóm có vai trò và quyền hạn khác nhau. Admin chịu trách nhiệm quản lý và kiểm soát hệ thống; Librarian thực hiện các nghiệp vụ liên quan đến quản lý sách và mượn trả; Member là người sử dụng các dịch vụ thư viện như tìm kiếm, xem thông tin sách, mượn sách và theo dõi lịch sử mượn trả. Việc nghiên cứu các nhóm người dùng giúp xác định rõ chức năng và quyền truy cập tương ứng cho từng vai trò.
- Thứ hai, nghiên cứu đối tượng sách và tài nguyên thư viện. Đề tài nghiên cứu cách lưu trữ và quản lý thông tin của sách như tên sách, tác giả, mã ISBN, thể loại, nhà xuất bản, năm xuất bản, số lượng, số lượng còn lại và trạng thái hoạt động. Việc quản lý tập trung các thông tin này giúp hệ thống có thể kiểm soát tình trạng sách và hỗ trợ người dùng tra cứu thông tin một cách thuận tiện.

- Thứ ba, nghiên cứu nghiệp vụ mượn và trả sách. Đây là một trong những nghiệp vụ trọng tâm của hệ thống. Đề tài nghiên cứu quy trình từ khi người dùng thực hiện mượn sách, kiểm tra điều kiện mượn, xác định số lượng sách còn lại, ghi nhận ngày mượn và hạn trả cho đến khi người dùng trả sách. Hệ thống đồng thời cập nhật trạng thái phiếu mượn và số lượng sách sau mỗi giao dịch.
- Thứ tư, nghiên cứu nghiệp vụ quản lý sách quá hạn và tiền phạt. Đề tài nghiên cứu cách xác định một phiếu mượn có quá hạn hay không dựa trên ngày trả thực tế và hạn trả. Khi sách được trả quá hạn, hệ thống thực hiện tính toán khoản tiền phạt dựa trên số ngày quá hạn và mức phạt được quy định. Đây là chức năng nhằm giảm bớt công việc tính toán thủ công và hạn chế sai sót trong quá trình quản lý.
- Thứ năm, nghiên cứu cơ chế xác thực và phân quyền. Đề tài nghiên cứu phương pháp xác thực tài khoản khi người dùng đăng nhập và cơ chế phân quyền dựa trên vai trò. Người dùng sau khi đăng nhập chỉ được phép truy cập các chức năng phù hợp với quyền hạn của mình. Nội dung này góp phần đảm bảo tính bảo mật và kiểm soát truy cập cho hệ thống.
- Thứ sáu, nghiên cứu kiến trúc và công nghệ xây dựng hệ thống. Đề tài nghiên cứu việc xây dựng một ứng dụng web với React và Vite ở phía Frontend, Node.js và Express.js ở phía Backend và MongoDB kết hợp Mongoose để quản lý dữ liệu. Hệ thống được tổ chức theo hướng tách biệt giữa giao diện người dùng, xử lý nghiệp vụ và cơ sở dữ liệu, giúp thuận tiện cho việc phát triển, kiểm thử và mở rộng.
- Thứ bảy, nghiên cứu quá trình phát triển và kiểm thử phần mềm. Đề tài nghiên cứu các bước từ phân tích yêu cầu, thiết kế hệ thống, xây dựng chức năng, tích hợp Frontend với Backend đến kiểm thử các API và chức năng trên giao diện. Việc kiểm thử tập trung vào các chức năng chính như đăng ký, đăng nhập, phân quyền, quản lý sách, mượn sách, trả sách và tính tiền phạt.

Như vậy, đối tượng nghiên cứu của đề tài bao gồm các nghiệp vụ quản lý thư viện, người dùng, sách, hoạt động mượn trả, cơ chế xác thực – phân quyền và các công nghệ được sử dụng để xây dựng hệ thống. Thông qua việc nghiên cứu các đối tượng trên, đề tài hướng đến việc xây dựng một ứng dụng quản lý thư viện phù hợp với các yêu cầu thực tế và có khả năng mở rộng trong tương lai.

#### **4. Phạm vi nghiên cứu**

Đề tài “Xây dựng ứng dụng quản lý thư viện” tập trung nghiên cứu và xây dựng một ứng dụng quản lý thư viện trên nền tảng web, phục vụ các nghiệp vụ cơ bản thường gặp trong quá trình quản lý sách và người dùng. Do giới hạn về thời gian, nguồn lực và phạm vi của đồ án môn học, đề tài tập trung vào những chức năng cốt lõi, có tính thực tế và phù hợp với quy mô của một thư viện trường học hoặc thư viện có quy mô vừa và nhỏ.

- Về phạm vi nghiệp vụ, hệ thống tập trung vào việc quản lý tài khoản người dùng, quản lý sách và quản lý hoạt động mượn – trả sách. Hệ thống hỗ trợ ba nhóm người dùng chính gồm Admin, Librarian và Member. Mỗi nhóm được phân quyền khác nhau nhằm đảm bảo người dùng chỉ có thể thực hiện các chức năng phù hợp với vai trò của mình.
- Đối với quản lý người dùng, phạm vi nghiên cứu bao gồm chức năng đăng ký tài khoản, đăng nhập, xác thực người dùng và phân quyền theo vai trò. Hệ thống lưu trữ các thông tin cơ bản của người dùng như họ tên, email, mật khẩu, số điện thoại, vai trò và trạng thái tài khoản. Các chức năng liên quan đến quản lý tài khoản được xây dựng nhằm phục vụ việc kiểm soát người sử dụng và bảo đảm an toàn khi truy cập hệ thống.
- Đối với quản lý sách, hệ thống tập trung vào các chức năng thêm, xem và cập nhật thông tin sách. Các thông tin được quản lý bao gồm tên sách, tác giả, ISBN, thể loại, nhà xuất bản, năm xuất bản, số lượng, số lượng sách có thể cho mượn, mô tả và trạng thái hoạt động. Hệ thống hỗ trợ kiểm soát số lượng sách còn lại để phục vụ cho quá trình mượn sách và hạn chế trường hợp cho mượn khi thư viện không còn sách khả dụng.

- Đối với quản lý mượn – trả sách, hệ thống nghiên cứu và triển khai quy trình từ lúc người dùng thực hiện yêu cầu mượn sách cho đến khi sách được trả lại. Khi phát sinh giao dịch mượn, hệ thống kiểm tra thông tin người dùng, trạng thái tài khoản, tình trạng sách và số lượng sách còn lại trước khi tạo phiếu mượn. Thông tin về ngày mượn, hạn trả và trạng thái mượn được lưu trữ để phục vụ việc theo dõi.
- Khi người dùng trả sách, hệ thống cập nhật ngày trả, trạng thái phiếu mượn và số lượng sách có sẵn. Trong trường hợp người dùng trả sách sau thời hạn quy định, hệ thống xác định số ngày quá hạn và tính tiền phạt tương ứng. Chức năng này nằm trong phạm vi nghiên cứu nhằm mô phỏng một nghiệp vụ thực tế của thư viện và giảm bớt việc xử lý thủ công.
- Về phạm vi công nghệ, đề tài sử dụng các công nghệ phát triển ứng dụng web gồm React và Vite để xây dựng giao diện người dùng, Node.js và Express.js để xây dựng Backend và cung cấp các RESTful API, MongoDB và Mongoose để lưu trữ và quản lý dữ liệu. Quá trình phát triển được thực hiện trên môi trường Visual Studio Code và các API được kiểm thử bằng Postman. Git/GitHub được sử dụng để quản lý mã nguồn trong quá trình phát triển.
- Về phạm vi người sử dụng, hệ thống được xây dựng hướng đến mô hình thư viện có ba nhóm đối tượng chính. Admin có quyền quản lý và kiểm soát các chức năng quan trọng của hệ thống; Librarian thực hiện các nghiệp vụ quản lý sách và mượn – trả; Member sử dụng các chức năng dành cho độc giả như xem thông tin sách, mượn sách và theo dõi các phiếu mượn của bản thân.
- Về phạm vi triển khai, hệ thống được xây dựng và kiểm thử chủ yếu trong môi trường phát triển cục bộ. Frontend và Backend được chạy trên các cổng riêng biệt và kết nối với cơ sở dữ liệu MongoDB. Việc triển khai thực tế trên hạ tầng máy chủ của một thư viện hoặc tích hợp với các

hệ thống quản lý thư viện hiện có chưa nằm trong phạm vi chính của đề tài.

Bên cạnh đó, đề tài chưa tập trung vào một số chức năng nâng cao như quản lý kho sách chuyên sâu, quản lý nhiều chi nhánh thư viện, thanh toán tiền phạt trực tuyến, gửi thông báo qua email hoặc SMS, quét mã vạch/QR Code, tích hợp máy đọc thẻ thư viện, báo cáo thống kê chuyên sâu và hệ thống gợi ý sách dựa trên hành vi người dùng. Những chức năng này có thể được xem xét và phát triển trong các giai đoạn tiếp theo.

Nhìn chung, phạm vi nghiên cứu của đề tài được giới hạn ở việc phân tích, thiết kế, xây dựng và kiểm thử một hệ thống quản lý thư viện web với các chức năng cốt lõi gồm quản lý người dùng, phân quyền, quản lý sách, mượn sách, trả sách và xử lý tiền phạt khi quá hạn. Việc giới hạn phạm vi giúp đề tài phù hợp với thời gian thực hiện đồ án, đồng thời đảm bảo các chức năng chính có thể được xây dựng, kiểm thử và đánh giá một cách tương đối đầy đủ.

## **5. Giới hạn của đề tài**

Do giới hạn về thời gian thực hiện, nguồn lực và phạm vi của đồ án môn học Công nghệ phần mềm, đề tài “Xây dựng ứng dụng quản lý thư viện” tập trung vào việc xây dựng một hệ thống quản lý thư viện ở mức cơ bản, đáp ứng các nghiệp vụ chính liên quan đến người dùng, sách và hoạt động mượn – trả sách. Đề tài chưa hướng đến việc xây dựng một hệ thống quản lý thư viện có quy mô lớn hoặc đáp ứng toàn bộ các nghiệp vụ chuyên sâu của các thư viện chuyên nghiệp.

- Về đối tượng sử dụng, hệ thống hiện tập trung vào ba nhóm người dùng gồm Admin, Librarian và Member. Mỗi nhóm được cấp các quyền khác nhau dựa trên vai trò. Đề tài chưa triển khai các mô hình phân quyền phức tạp như phân quyền theo từng chức năng chi tiết, phân quyền theo phòng ban hoặc theo nhiều cấp quản lý.
- Về quản lý sách, hệ thống tập trung vào các thông tin cơ bản như tên sách, tác giả, ISBN, thể loại, nhà xuất bản, năm xuất bản, số lượng và

trạng thái sách. Đề tài chưa triển khai các nghiệp vụ quản lý kho nâng cao như quản lý vị trí chính xác của từng bản sách, mã vạch cho từng bản sao, nhập – xuất kho, thanh lý sách, kiểm kê định kỳ hoặc quản lý nhiều kho sách tại nhiều chi nhánh khác nhau.

- Về nghiệp vụ mượn – trả, hệ thống chủ yếu xử lý quy trình mượn sách, xác định hạn trả, trả sách và cập nhật số lượng sách. Hệ thống có hỗ trợ xác định trường hợp trả sách quá hạn và tính tiền phạt. Tuy nhiên, đề tài chưa triển khai các nghiệp vụ phức tạp như gia hạn sách nhiều lần, đặt trước sách, xử lý danh sách chờ mượn, giới hạn số sách được mượn theo từng nhóm thành viên hoặc các quy định mượn sách khác nhau theo từng loại tài liệu.
- Về thanh toán, hệ thống mới dừng ở việc xác định và ghi nhận số tiền phạt khi người dùng trả sách quá hạn. Chức năng thanh toán trực tuyến thông qua ngân hàng, ví điện tử hoặc các cổng thanh toán chưa được triển khai. Việc thu tiền phạt trong thực tế vẫn cần được thực hiện thông qua quy trình quản lý riêng của thư viện.
- Về thông báo, hệ thống chưa tích hợp các dịch vụ gửi email, SMS hoặc thông báo đẩy đến người dùng. Vì vậy, hệ thống chưa tự động gửi thông báo khi sách sắp đến hạn trả, đã quá hạn hoặc khi phát sinh các khoản tiền phạt. Đây là một chức năng có thể được bổ sung trong quá trình phát triển tiếp theo.
- Về tìm kiếm và thống kê, hệ thống chủ yếu phục vụ việc xem và quản lý thông tin sách và các giao dịch mượn – trả. Các chức năng phân tích dữ liệu chuyên sâu như thống kê số lượng sách được mượn theo từng khoảng thời gian, phân tích xu hướng đọc sách, thống kê doanh thu từ tiền phạt hoặc xây dựng biểu đồ phân tích hoạt động thư viện chưa nằm trong phạm vi chính của đề tài.
- Về tích hợp thiết bị và công nghệ bên ngoài, đề tài chưa triển khai các thiết bị hoặc công nghệ chuyên dụng như máy quét mã vạch, QR Code, thẻ RFID hoặc máy đọc thẻ thư viện. Các thông tin về sách và

người dùng hiện được nhập và xử lý thông qua giao diện của ứng dụng.

- Về triển khai hệ thống, ứng dụng chủ yếu được phát triển và kiểm thử trong môi trường phục vụ đồ án. Hệ thống chưa được triển khai ở quy mô lớn với nhiều máy chủ, cơ chế cân bằng tải, sao lưu dữ liệu tự động, giám sát hệ thống hoặc các giải pháp đảm bảo khả năng hoạt động liên tục. Do đó, khả năng chịu tải khi có số lượng lớn người dùng truy cập đồng thời chưa được đánh giá chuyên sâu.
- Cuối cùng, đề tài tập trung vào việc xây dựng một sản phẩm phần mềm đáp ứng các nghiệp vụ cốt lõi và minh họa quy trình phát triển phần mềm, do đó chưa đặt mục tiêu thay thế hoàn toàn một hệ thống quản lý thư viện chuyên nghiệp. Một số chức năng nâng cao như quản lý nhiều chi nhánh, đặt trước sách, thông báo tự động, thanh toán trực tuyến, quản lý mã vạch/QR Code, RFID và phân tích dữ liệu có thể được nghiên cứu và phát triển trong tương lai.

Những giới hạn trên giúp đề tài tập trung nguồn lực vào các chức năng quan trọng nhất, đảm bảo hệ thống có thể được phân tích, thiết kế, xây dựng và kiểm thử trong phạm vi thời gian của đồ án. Đồng thời, các giới hạn này cũng tạo cơ sở để xác định hướng phát triển của hệ thống trong tương lai, nhằm từng bước hoàn thiện ứng dụng và nâng cao khả năng đáp ứng nhu cầu thực tế của thư viện.

---

## CHƯƠNG 1: TỔNG QUAN

Trong những năm gần đây, sự phát triển của công nghệ thông tin đã tạo ra nhiều thay đổi trong cách thức quản lý và vận hành các tổ chức, trong đó có hoạt động quản lý thư viện. Thư viện không chỉ là nơi lưu trữ và cung cấp sách mà còn phải quản lý một lượng lớn thông tin liên quan đến tài liệu, người dùng, hoạt động mượn – trả và tình trạng sử dụng tài nguyên. Nếu các nghiệp vụ này được thực hiện hoàn toàn bằng phương pháp thủ công, nhân viên thư viện có thể gặp nhiều khó khăn trong việc cập nhật, tra cứu và kiểm soát dữ liệu. Vì vậy, việc ứng dụng phần mềm vào quản lý thư viện là một hướng tiếp cận phù hợp nhằm nâng cao hiệu quả quản lý và phục vụ người dùng.

Đề tài “Xây dựng ứng dụng quản lý thư viện” được thực hiện với mục tiêu xây dựng một hệ thống ứng dụng web hỗ trợ các nghiệp vụ cơ bản của thư viện. Hệ thống cho phép quản lý tập trung thông tin về sách, người dùng và các hoạt động mượn – trả sách. Thay vì phải ghi nhận và kiểm tra thông tin bằng sổ sách hoặc các phương pháp thủ công, dữ liệu được lưu trữ trong cơ sở dữ liệu và được xử lý thông qua các chức năng của hệ thống. Điều này giúp giảm thời gian xử lý, hạn chế sai sót và tạo điều kiện thuận lợi cho việc tra cứu thông tin.

Hệ thống được thiết kế với ba nhóm người dùng chính gồm Admin, Librarian và Member. Mỗi nhóm người dùng có quyền hạn khác nhau dựa trên vai trò trong thư viện. Admin có quyền quản lý và kiểm soát các chức năng quan trọng của hệ thống. Librarian chịu trách nhiệm thực hiện các nghiệp vụ quản lý sách và hỗ trợ quá trình mượn – trả. Member là người sử dụng dịch vụ thư viện, có thể xem thông tin sách, thực hiện mượn sách và theo dõi các giao dịch mượn – trả của bản thân. Việc phân chia quyền hạn giúp hệ thống kiểm soát quyền truy cập và hạn chế việc người dùng thực hiện các thao tác không phù hợp với vai trò.

Một trong những chức năng trọng tâm của hệ thống là quản lý sách. Hệ thống cho phép lưu trữ và cập nhật các thông tin cơ bản của sách như tên sách, tác giả, ISBN, thể loại, nhà xuất bản, năm xuất bản, số lượng, số lượng có sẵn và trạng thái hoạt động. Khi có hoạt động mượn hoặc trả sách, số lượng sách có sẵn được cập nhật tương ứng. Nhờ đó, nhân viên thư viện có thể theo dõi tình trạng tài nguyên và xác định khả năng cho mượn của từng đầu sách.

Bên cạnh đó, hệ thống cung cấp chức năng quản lý mượn – trả sách nhằm mô phỏng quy trình nghiệp vụ thực tế. Khi người dùng yêu cầu mượn sách, hệ thống kiểm tra thông tin tài khoản, trạng thái hoạt động của người dùng, thông tin sách và số lượng sách còn lại. Nếu các điều kiện phù hợp, hệ thống tạo phiếu mượn và ghi nhận các thông tin cần thiết như ngày mượn, hạn trả và trạng thái phiếu mượn. Khi sách được trả, hệ thống cập nhật ngày trả, trạng thái giao dịch và số lượng sách có sẵn.

Đặc biệt, hệ thống có khả năng xử lý trường hợp trả sách quá hạn và tính tiền phạt. Dựa trên hạn trả và ngày trả thực tế, hệ thống xác định số ngày quá hạn và tính khoản tiền phạt tương ứng theo quy định được thiết lập. Chức năng này giúp giảm công việc tính toán thủ công cho nhân viên thư viện và đảm bảo việc xử lý tiền phạt được thực hiện thống nhất.

Về mặt kiến trúc, hệ thống được xây dựng theo mô hình ứng dụng web với sự phân tách giữa Frontend, Backend và Database. Phần Frontend được xây dựng bằng React kết hợp với Vite, chịu trách nhiệm hiển thị giao diện và tiếp nhận thao tác của người dùng. Phần Backend sử dụng Node.js và Express.js để xử lý các yêu cầu, thực hiện nghiệp vụ và cung cấp các RESTful API cho Frontend. Dữ liệu được lưu trữ trong MongoDB, kết hợp với Mongoose để định nghĩa mô hình dữ liệu và thực hiện các thao tác với cơ sở dữ liệu.

Quy trình hoạt động tổng thể của hệ thống có thể được mô tả như sau: người dùng truy cập ứng dụng thông qua giao diện web và thực hiện đăng ký hoặc đăng nhập. Sau khi xác thực thành công, hệ thống xác định vai trò của người dùng và cấp quyền truy cập tương ứng. Người dùng có thể thực hiện các chức năng được cho phép, chẳng hạn như xem sách, mượn sách hoặc theo dõi lịch sử mượn. Các yêu cầu từ Frontend được gửi đến Backend thông qua API. Backend thực hiện kiểm tra dữ liệu, xác thực quyền truy cập, xử lý nghiệp vụ và tương tác với MongoDB trước khi trả kết quả về cho Frontend.

Trong quá trình xây dựng, hệ thống được kiểm thử thông qua các trường hợp khác nhau nhằm đánh giá tính chính xác của các chức năng. Các API được kiểm tra bằng Postman, trong khi các chức năng trên giao diện được kiểm tra trực tiếp trên ứng dụng web. Nội dung kiểm thử tập trung vào các chức năng quan trọng như đăng

ký, đăng nhập, phân quyền, quản lý sách, mượn sách, trả sách và xử lý sách quá hạn.

Nhìn chung, đề tài hướng đến việc xây dựng một ứng dụng quản lý thư viện có các chức năng cốt lõi, giao diện dễ sử dụng và kiến trúc có khả năng mở rộng. Mặc dù hệ thống vẫn còn giới hạn về một số chức năng nâng cao như thanh toán trực tuyến, thông báo tự động, quản lý mã vạch hoặc RFID, nhưng sản phẩm đã tập trung giải quyết những nghiệp vụ cơ bản và cần thiết trong quản lý thư viện. Đồng thời, quá trình thực hiện đề tài giúp vận dụng các kiến thức về phân tích yêu cầu, thiết kế hệ thống, cơ sở dữ liệu, lập trình Web, xác thực, phân quyền và kiểm thử phần mềm vào một bài toán thực tế.

---

## CHƯƠNG 2: NGHIÊN CỨU LÝ THUYẾT

### 2.1 Tổng quan về Công nghệ phần mềm

#### 2.1.1 Khái niệm

Công nghệ phần mềm (*software engineering*) là sự áp dụng một cách tiếp cận có hệ thống, có kỷ luật, và định lượng được cho việc phát triển, sử dụng và bảo trì phần mềm. Ngành học kỹ sư phần mềm bao trùm kiến thức, các công cụ, và các phương pháp cho việc định nghĩa yêu cầu phần mềm, và thực hiện các tác vụ thiết kế, xây dựng, kiểm thử (*software testing*), và bảo trì phần mềm. Kỹ sư phần mềm còn sử dụng kiến thức của các lĩnh vực như kỹ thuật máy tính, khoa học máy tính, quản lý, toán học, quản lý dự án, quản lý chất lượng, công thái học phần mềm (*software ergonomics*), và kỹ sư hệ thống (*systems engineering*)

#### 2.1.2 Quy trình phát triển phần mềm

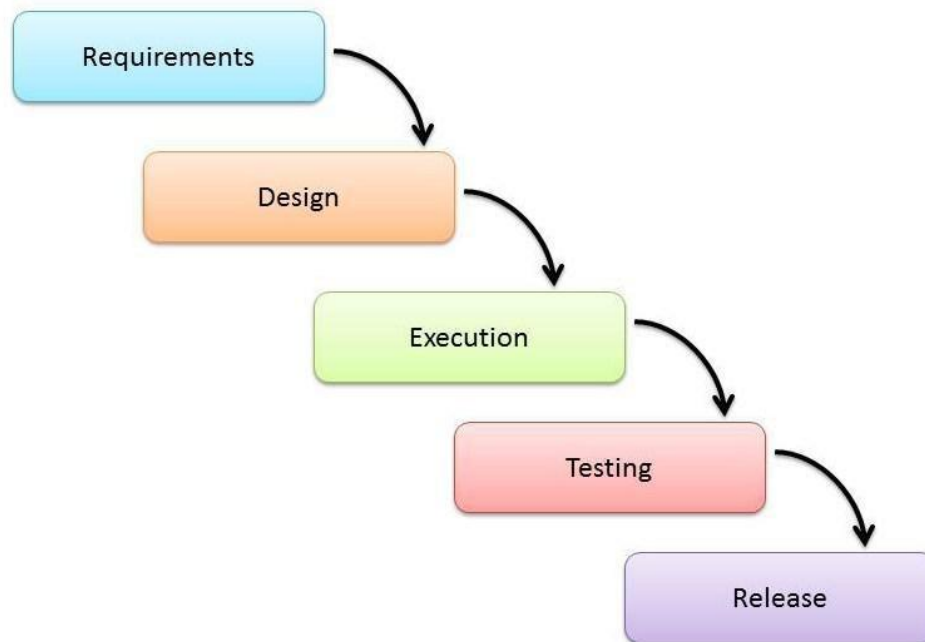
Quy trình phát triển phần mềm là một cấu trúc bao gồm tập hợp các thao tác và các kết quả tương quan sử dụng trong việc phát triển để sản xuất ra một sản phẩm phần mềm.

Các hoạt động cơ bản của quy trình phát triển phần mềm:

- Đặc tả phần mềm: Định nghĩa được các chức năng, điều kiện hoạt động của phần mềm.
- Phát triển phần mềm: Là quá trình xây dựng các đặc tả.
- Đánh giá phần mềm: Phần mềm phải được đánh giá để chắc chắn rằng ít nhất có thể thực hiện những gì mà tài liệu đặc tả yêu cầu.
- Tiến hóa phần mềm: Đây là quá trình hoàn thiện các chức năng cũng như giao diện để ngày càng hoàn thiện phần mềm cũng như các yêu cầu đưa ra từ phía khách hàng.

#### 2.1.3 Các mô hình phát triển phần mềm

##### 1. Mô hình thác nước – Waterfall model



*Hình 2.1 Mô hình thác nước – Waterfall model*

Mô hình thác nước (tiếng Anh: waterfall model) là một mô hình của quy trình phát triển phần mềm, trong đó quy trình phát triển trông giống như một dòng chảy, với các pha được thực hiện theo trật tự nghiêm ngặt và không có sự quay lui hay nhảy vượt pha là: phân tích yêu cầu, thiết kế, triển khai thực hiện, kiểm thử, liên kết và bảo trì.

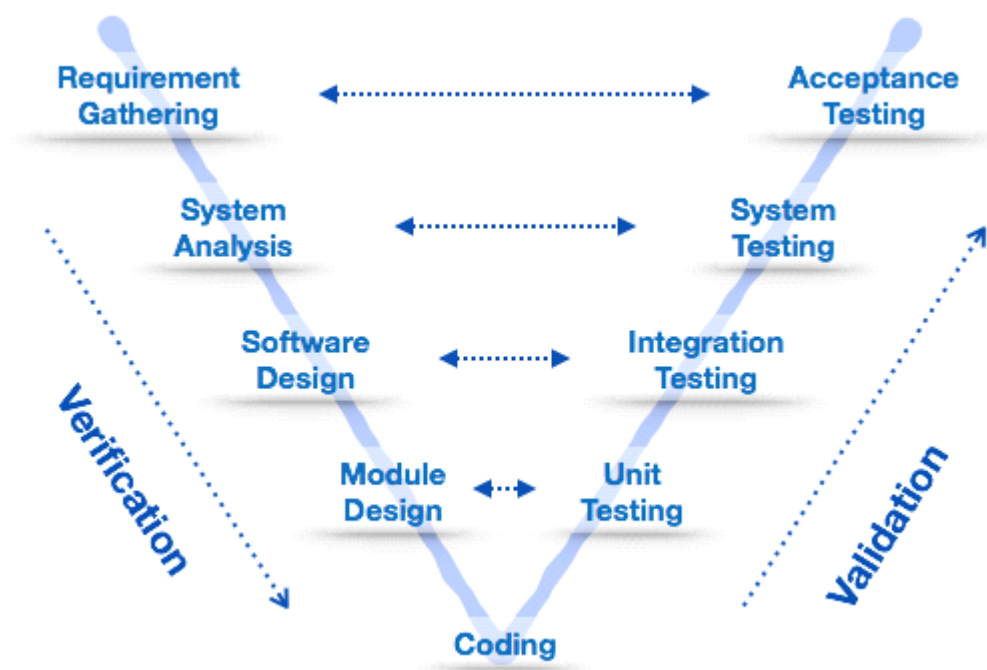
Các giai đoạn của mô hình thác nước:

- **Thu thập yêu cầu** (Requirement gathering) : Đây là giai đoạn xác định các yêu cầu chức năng và phi chức năng mà hệ thống phần mềm cần có. Kết quả của giai đoạn này là bản tài liệu đặc tả yêu cầu. Tài liệu này sẽ là nền tảng cho những giai đoạn tiếp theo cho đến cuối dự án.
- **Phân tích hệ thống** ( System Analysis): Là giai đoạn định ra làm thế nào để hệ thống phần mềm đáp ứng đúng yêu cầu của khách hàng. Giai đoạn này thực hiện phân tích, thiết kế hệ thống phần mềm.
- **Coding**: Là giai đoạn thực hiện sản phẩm dựa trên đặc tả yêu cầu và tài liệu thiết kế module.
- **Testing**: Tester sẽ nhận sản phẩm từ dev và thực hiện kiểm thử cho nhóm các thành phần và kiểm thử hệ thống. Khâu kiểm thử

cuối cùng sẽ là Kiểm thử chấp nhận, giai đoạn này còn có sự tham gia của khách hàng.

- **Implementation:** Triển khai hệ thống ra môi trường của khách hàng.
- **Operations & Maintenance:** Đây là giai đoạn cài đặt, cấu hình và đào tạo cho khách hàng. Giai đoạn này sửa chữa những lỗi của sản phẩm (nếu có) và phát triển những thay đổi mới được khách hàng yêu cầu.
- **Áp dụng:** Thường được áp dụng cho các dự án không thường xuyên bị thay đổi về yêu cầu.
- **Đặc điểm:**
  - + Ưu điểm: Dễ sử dụng, dễ tiếp cận Các giai đoạn và hoạt động được xác định rõ ràng Xác nhận ở từng giai đoạn, đảm bảo phát hiện sớm các lỗi
  - + Nhược điểm: Rất khó để quay lại giai đoạn nào khi nó đã kết thúc Ít tính linh hoạt và phạm vi điều chỉnh của nó khá là khó khăn, tốn kém.

## 2. Mô hình chữ V – V-Shaped model



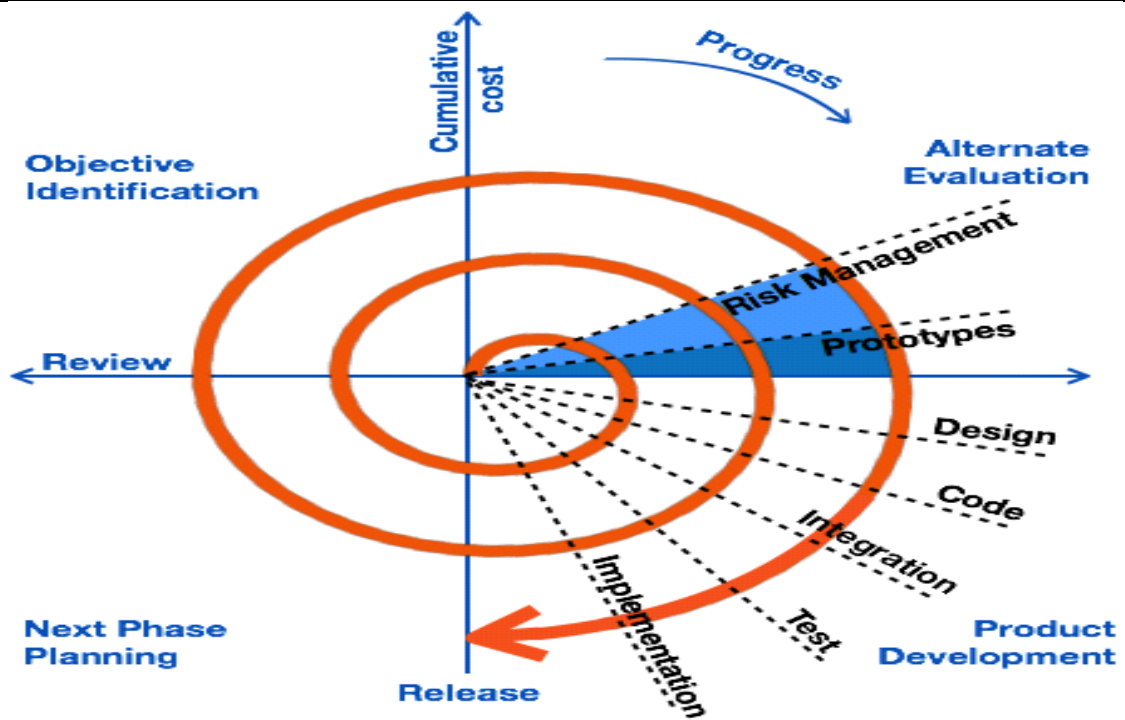
Hình 2.2 Mô hình V-V-Shaped Model

Mô hình V hiện nay là một trong những quy trình phát triển phần mềm được sử dụng rộng rãi nhất. Trong mô hình V việc thực hiện kiểm tra được diễn ra ngay từ giai đoạn lấy yêu cầu. V mô hình cũng được gọi là mô hình xác minh (verification) và mô hình xác nhận (validation).

Để hiểu được mô hình V, trước hết chúng ta hãy hiểu xác minh (verification) và xác nhận hợp lệ (validation) trong phần mềm là gì.

- **Xác minh (verification) :** Xác minh là một kỹ thuật phân tích tĩnh. Trong kiểm thử, kỹ thuật này được thực hiện mà không phải chạy code. Nó bao gồm một số hoạt động như xem lại (review), kiểm tra (inspection) và kiểm tra từ đầu tới cuối (walkthrough).
- **Xác nhận (validation):** Xác nhận là một kỹ thuật phân tích động, trong đó việc kiểm thử được thực hiện bằng cách thực hiện code. Ví dụ bao gồm kỹ thuật kiểm tra chức năng (function) và phi chức năng (non-function).
- **Áp dụng:** Yêu cầu phần mềm phải xác định rõ ràng Công nghệ phần mềm và các công cụ phải được tìm hiểu kỹ
- **Đặc điểm:**
  - + Ưu điểm: Đơn giản dễ sử dụng Phân phối cụ thể theo mỗi giai đoạn Thực hiện verification và validation sớm trong mỗi giai đoạn phát triển
  - + Nhược điểm: Phạm vi điều chỉnh khá là khó khăn và tốn kém. Trong mô hình V, các hoạt động phát triển và đảm bảo chất lượng được thực hiện đồng thời. Không có pha rời rạc được gọi là kiểm thử, thay vào đó kiểm thử được bắt đầu ngay từ giai đoạn lấy yêu cầu. Các hoạt động xác minh và xác nhận đi liền với nhau.

### 3. Mô hình xoắn ốc – Spiral model



Hình 2.3 Mô hình xoắn ốc – Spiral model

Mô hình xoắn ốc (tiếng Anh: spiral model) là quy trình phát triển định hướng rủi ro cho các dự án phần mềm. Kết hợp của thế mạnh của các mô hình khác và giải quyết khó khăn của các mô hình trước còn tồn tại. Dựa trên các mô hình rủi ro riêng biệt của mỗi dự án, mô hình xoắn ốc đưa ra cách áp dụng các yếu tố của một hoặc nhiều mô hình xử lý, chẳng hạn như mô hình gia tốc, mô hình thác nước hoặc mô hình tạo mẫu tiến hóa.

Các giai đoạn của mô hình xoắn ốc:

- **Lập kế hoạch – Planning phase:** Thu thập, phân tích yêu cầu từ của dự án từ phía khách hàng. Bao gồm các công việc: Ước lượng chi phí (estimating cost), lên lịch trình thực hiện dự án (shedule-master), xác định số lượng nhân lực, môi trường làm việc (identifying necessary resources and work environment), tìm hiểu yêu cầu hệ thống (requirements) từ đó đưa ra các tài liệu đặc tả (Bussiness Requirement Specifications và System Requirement specifications) để phục vụ cho việc trao đổi giữa khách hàng và phân tích hệ thống sau này.
- **Phân tích rủi ro – Risk analysis phase:** Một quá trình phân tích sẽ được thực hiện để xác định rủi ro và đưa ra các giải pháp thay thế. Một prototype sẽ được tạo ra ở cuối giai đoạn phân tích rủi ro. Nếu có bất kỳ rủi ro nào được

tìm thấy trong quá trình này thì các giải pháp thay thế sẽ được đề xuất và thực hiện.

- **Thực thi kỹ thuật – Engineering phase:** Đây là giai đoạn mà dự án được các dev tiến hành code, các tester tiến hành test và deploying software trên trang web của khách hàng.
- **Đánh giá – Evaluation phase:** Khách hàng sẽ tham gia vào giai đoạn này để đánh giá công việc, sản phẩm và đảm bảo rằng sản phẩm đáp ứng tất cả các yêu cầu đã đặt ra trước đó. Nếu có bất kỳ yêu cầu thay đổi nào từ khách hàng, các giai đoạn sẽ được lặp lại. Đây là giai đoạn quan trọng vì cần có sự phản hồi của khách hàng về sản phẩm trước khi sản phẩm được release.
- **Áp dụng:** Thường được sử dụng cho các ứng dụng lớn và các hệ thống được xây dựng theo các giai đoạn nhỏ hoặc theo các phân đoạn
- **Đặc điểm:**
  - + Ưu điểm: Estimates (i.e. budget, schedule, etc.) trở nên thực tế hơn như là một quy trình làm việc, bởi vì những vấn đề quan trọng đã được phát hiện sớm hơn. Có sự tham gia sớm của developers Quản lý rủi ro và phát triển hệ thống theo phase
  - + Nhược điểm: Chi phí cao và thời gian dài để có sản phẩm cuối cùng Phải có kỹ năng tốt để đánh giá rủi ro và giả định.

#### 4. Mô hình Agile – Agile Model

Agile là một tập hợp các nguyên lý dành cho phát triển phần mềm, trong đó khuyến khích việc lập kế hoạch thích ứng, phát triển tăng dần, chuyển giao sớm, và cải tiến liên tục. Agile cũng chủ trương thích ứng nhanh chóng với các thay đổi. Những nguyên lý này được chia sẻ trong Tuyên ngôn Phát triển Phần mềm Linh hoạt và 12 Nguyên lý phía sau.

Agile không định nghĩa một phương pháp cụ thể để đạt được những điều này, nhưng lại có nhiều phương pháp phát triển phần mềm khác nhau thỏa mãn và hướng theo các tiêu chí đó.

Mục đích của các phương pháp Agile là giúp các doanh nghiệp đạt được sự linh hoạt (Agility), từ đó nâng cao sức cạnh tranh và phát triển bền vững. Các phương pháp Agile đã thay đổi diện mạo thế giới không chỉ trong Phát triển phần mềm mà còn đang thể hiện giá trị trong các lĩnh vực khác như Marketing (Agile Marketing), giáo dục (EduScrum, Lean Edu, v.v.), thiết kế (Lean UX, Design Thinking), khởi nghiệp (Lean Startup) và Phần cứng.

Áp dụng: Có thể được sử dụng với bất kỳ loại hình dự án nào, nhưng nó cần sự tham gia và tính tương tác của khách hàng. Ngoài ra, nó có thể được sử dụng khi khách hàng yêu cầu chức năng sẵn sàng trong khoảng thời gian ngắn ( 3 tuần )

Đặc điểm:

- + Ưu điểm: Giảm thời gian cần thiết để tận dụng một số tính năng của hệ thống  
Kết quả cuối cùng là phần mềm chất lượng cao trong thời gian ít nhất có thể và sự hài lòng của khách hàng
- + Nhược điểm: Phụ thuộc vào kỹ năng của người phát triển phần mềm  
Scalability Tài liệu được thực hiện ở giai đoạn sau

## 5.Mô hình Scrum

Scrum là một quy trình phát triển phần mềm thuộc họ agile.

Là một quy trình phát triển phần mềm theo mô hình linh hoạt (agile). Với nguyên tắc chủ đạo là chia nhỏ phần mềm cần sản xuất ra thành các phần nhỏ để phát triển (các phần nhỏ này phải độc lập và Release được), lấy ý kiến khách hàng và thay đổi cho phù hợp ngay trong quá trình phát triển để đảm bảo sản phẩm release đáp ứng những gì khách hàng mong muốn. Scrum chia dự án thành các vòng lặp phát triển gọi là các sprint. Mỗi sprint thường mất 2- 4 tuần (30 ngày) để hoàn thành. Nó rất phù hợp cho những dự án có nhiều sự thay đổi và yêu cầu tốc độ cao.

Ưu điểm: Một người có thể làm nhiều việc ví dụ như dev có thể test Phát hiện lỗi sớm hơn rất nhiều so với các phương pháp truyền thống Khách hàng nhanh chóng thấy được sản phẩm qua đó đưa ra phản hồi sớm. Có khả năng áp dụng được cho những dự án mà yêu cầu khách hàng không rõ ràng ngay từ đầu.

Nhược điểm: Trình độ của nhóm là có một kỹ năng nhất định Phải có sự hiểu biết về mô hình agile . Khó khăn trong việc xác định ngân sách và thời gian. Luôn

nghe ý kiến phản hồi từ khách hàng và thay đổi theo nên thời gian sẽ kéo dài khi có quá nhiều yêu cầu thay đổi từ khách hàng. Vai trò của PO (Product Owner) rất quan trọng, PO là người định hướng sản phẩm. Nếu PO làm không tốt sẽ ảnh hưởng đến kết quả chung.

## **2.2 Hệ thống quản lý thư viện**

### **2.2.1 Giới thiệu về Hệ thống quản lý thư viện**

Hệ thống Quản lý Thư viện là một ứng dụng phần mềm được phát triển để giúp các thư viện tổ chức và quản lý tài liệu, sách, báo, tạp chí và tài liệu tham khảo khác một cách hiệu quả. Hệ thống này sẽ cung cấp một giao diện thân thiện cho người dùng để tìm kiếm, mượn và trả sách, theo dõi thông tin về độc giả, quản lý kho sách, và thực hiện các chức năng quản lý khác.

### **2.2.2 Mục tiêu của hệ thống**

Mục tiêu chính của Hệ thống Quản lý Thư viện là:

- + Tối ưu hóa quá trình quản lý tài liệu và sách trong thư viện.
- + Đảm bảo tính chính xác và minh bạch trong việc theo dõi thông tin về độc giả và mượn/trả sách.
- + Cung cấp giao diện thân thiện và dễ sử dụng cho cả nhân viên thư viện và độc giả.
- + Tối ưu hóa việc tìm kiếm tài liệu và sách thông qua hệ thống phân loại và tìm kiếm thông minh.
- + Tăng cường bảo mật thông tin độc giả và tài liệu quan trọng.

### **2.2.3 Chức năng của hệ thống**

- Thêm đầu sách: Hệ thống cho phép nhân viên kho thêm một đầu sách, bao gồm các
  - thông tin tên đầu sách, tác giả, năm xuất bản, nhà xuất bản, giá bìa, số lượng, vị trí, mô tả. Hệ thống sẽ cập nhật đầu sách mới vào CSDL.
- Xóa đầu sách: Hệ thống cho phép nhân viên kho xóa đầu sách trong CSDL.
- Sửa thông tin đầu sách: Hệ thống cho phép nhân viên kho sửa thông tin về tên đầu sách, tác giả, năm xuất bản, nhà xuất bản, số lượng, mô tả của một

- đầu sách trong CSDL. Hệ thống sẽ cập nhật thông tin mới của đầu
- sách đó trong CSDL.
- Thêm độc giả mới: Nhân viên trực có thể thêm độc giả mới tại thư viện qua hệ thống. Thông tin cá nhân của độc giả sẽ được lưu trữ, bao gồm mã độc giả,
- tên độc giả, email, số điện thoại...
- Quản lý Mượn/Trả sách: Hệ thống sẽ cung cấp các chức năng, và giao diện Mượn sách và trả sách, cũng như quản lý các sách chưa trả và đến hạn cho nhân viên
- trực có thể quản lý việc mượn trả sách của độc giả.
- Tìm kiếm đầu sách: Hệ thống cung cấp chức năng tìm kiếm đầu sách cho người dùng hệ thống để có thể quản lý, tìm kiếm đầu sách. Hệ thống sẽ theo dõi số
- lượng sách còn lại trong kho và tự động cập nhật khi có sách mới
- được thêm vào hoặc độc giả trả sách.
- Sửa thông tin cá nhân: Người dùng có thể sửa thông tin về số điện thoại hoặc địa chỉ, hoặc email của mình tại thư viện qua hệ thống. Hệ thống sẽ cập
- nhật các thông tin mới của độc giả trong CSDL.
- Thêm sửa xóa thông tin nhân viên: Quản lý có thể quản lý nhân viên bằng cách thêm sửa xóa thông tin nhân viên, Hệ thống sẽ cập nhật thông tin về các nhân viên trong CSDL.

#### **2.2.4 Quản lý và bảo mật**

- Quản lý dữ liệu: Hệ thống sẽ cung cấp giao diện cho nhân viên thư viện để thêm, sửa đổi và xóa thông tin về sách, độc giả và các dữ liệu khác.
- Bảo mật Thông tin: Dữ liệu độc giả và thông tin về sách sẽ được bảo mật thông qua việc áp dụng cơ chế xác thực và phân quyền truy cập. Chỉ những người dùng có quyền mới có thể truy cập vào các chức năng quản lý

#### **2.2.5 Mô tả chi tiết hoạt động và quan hệ giữa các đối tượng**

- Thư viện được quản lý bởi nhiều nhân viên, thông tin nhân viên bao gồm (mã nhân viên, chức vụ, họ và tên, số điện thoại)

- Việc thêm, sửa thông tin, xóa đầu sách, quản lý mượn trả sách, quản lý độc giả được quản lý bởi thủ thư
- Thư viện có nhiều đầu sách, mỗi đầu sách gồm các thông tin ( Mã đầu sách, tên đầu sách, tác giả, năm xuất bản,nhà xuất bản, số lượng, mô tả), mỗi đầu sách có 1 mã đầu sách để phân biệt với các đầu sách khác, mỗi đầu sách có thể được mượn nhiều lần khác nhau bởi nhiều độc giả khác nhau( trừ khi trong số lượng đầu sách trong kho hết)
- Mỗi đầu sách được xuất bản bởi một nhà xuất bản theo năm. Mỗi nhà xuất bản có thể sản xuất nhiều đầu sách.
- Mỗi đầu sách được viết bởi một hoặc nhiều tác giả, mỗi tác giả có thể viết nhiều đầu sách.
- Mỗi bạn đọc có một thẻ độc giả chứa (Mã độc giả, tên độc giả, số điện thoại,email) của bạn đọc đó
- Thời gian tối đa mượn 1 quyển sách là 1 tuần kể từ ngày mượn quyển đó, nếu trả sau thời hạn này thì sẽ bị phạt 20.000 VNĐ giá trị bìa sách.
- Mỗi lần trả sách có thể trả một phần hoặc toàn bộ số lượng sách đang mượn
- Khi mượn sách mới, nhân viên trực vẫn xem được danh sách các sách mà một độc giả đã mượn và trả rồi hoặc chưa trả trước đây.

## **2.3 Tổng quan về NodeJS**

### **2.3.1 NodeJS là gì?**

NodeJS là một mã nguồn được xây dựng dựa trên nền tảng Javascript V8 Engine, nó được sử dụng để xây dựng các ứng dụng web như các trang video clip, các forum và đặc biệt là trang mạng xã hội phạm vi hẹp. NodeJS là một mã nguồn mở được sử dụng rộng bởi hàng ngàn lập trình viên trên toàn thế giới. NodeJS có thể chạy trên nhiều nền tảng hệ điều hành khác nhau từ WIndow cho tới Linux, OS X nên đó cũng là một lợi thế. NodeJS cung cấp các thư viện phong phú ở dạng Javascript Module khác nhau giúp đơn giản hóa việc lập trình và giảm thời gian ở mức thấp nhất.

---

### 2.3.2 Các đặc tính của NodeJS

- Không đồng bộ: Tất cả các API của NodeJS đều không đồng bộ (non-blocking), nó chủ yếu dựa trên nền của NodeJS Server và chờ đợi Server trả dữ liệu về. Việc di chuyển máy chủ đến các API tiếp theo sau khi gọi và cơ chế thông báo các sự kiện của Node.js giúp máy chủ để có được một phản ứng từ các cuộc gọi API trước (Realtime).
- Chạy rất nhanh: NodeJS được xây dựng dựa vào nền tảng V8 Javascript Engine nên việc thực thi chương trình rất nhanh.
- Đơn luồng nhưng khả năng mở rộng cao: Node.js sử dụng một mô hình luồng duy nhất với sự kiện lặp. cơ chế tổ chức sự kiện giúp các máy chủ để đáp ứng một cách không ngăn chặn và làm cho máy chủ cao khả năng mở rộng như trái ngược với các máy chủ truyền thống mà tạo ra hạn chế để xử lý yêu cầu. Node.js sử dụng một chương trình đơn luồng và các chương trình tương tự có thể cung cấp dịch vụ cho một số lượng lớn hơn nhiều so với yêu cầu máy chủ truyền thống như Apache HTTP Server.
- Không đệm:  
NodeJS không đệm bất kỳ một dữ liệu nào và các ứng dụng này chủ yếu là đầu ra dữ liệu.
- Có giấy phép: NodeJS đã được cấp giấy phép bởi MIT License.

## 2.4 Tổng quan về React

### 2.4.1 React là gì?

React là một thư viện JavaScript mã nguồn mở được tạo bởi Facebook (nay là Meta), chuyên dùng để thiết kế và xây dựng giao diện người dùng (User Interface - UI), đặc biệt hiệu quả cho các ứng dụng một trang (Single Page Application). React giúp các lập trình viên tạo ra các thành phần giao diện tương tác phức tạp một cách dễ dàng và mượt mà trên cả máy tính và điện thoại.

React là một thư viện (library), không phải là một framework. Điều này có nghĩa là React chỉ tập trung giải quyết vấn đề ở lớp View (giao diện hiển thị) trong mô hình MVC (Model-View-Controller). Nó cho phép lập trình viên tự do lựa chọn các công cụ khác để tích hợp vào dự án như quản lý trạng thái hay định tuyến, thay vì bị ràng buộc bởi một cấu trúc chặt chẽ như Angular.

---

### 2.4.2. Các khái niệm cốt lõi của React

#### - Component (Thành phần):

- + Là khối xây dựng cơ bản nhất của React. Mỗi Component là một phần UI độc lập, có thể tái sử dụng (như Button, Header, Sidebar).

- + Được viết dưới dạng Functional Component (sử dụng hàm JavaScript) hoặc Class Component (ít dùng hơn hiện nay).

#### - JSX (JavaScript XML)

- + Cú pháp mở rộng của JavaScript, cho phép bạn viết HTML trực tiếp bên trong mã JavaScript.

- + Giúp giao diện trực quan và dễ đọc hơn.

#### - Props (Properties)

- + Dữ liệu truyền từ Component cha xuống Component con (theo hướng 1 chiều).

- + Props mang tính chất read-only (chỉ đọc) và không thể bị sửa đổi bởi Component con nhận nó.

#### - State (Trạng thái)

- + Dữ liệu nội bộ nằm bên trong một Component, có thể thay đổi theo thời gian (ví dụ: khi người dùng nhấp chuột, nhập dữ liệu).

- + Khi State thay đổi, Component sẽ tự động được render lại (re-render) để cập nhật giao diện.

#### - Virtual DOM (DOM ảo)

- + Bản sao nhẹ của DOM thực tế nằm trong bộ nhớ.

- + Khi dữ liệu thay đổi, React sẽ so sánh Virtual DOM mới với Virtual DOM cũ (quá trình Diffing) và chỉ cập nhật đúng phần thay đổi lên DOM thực tế, giúp tăng hiệu năng đáng kể.

#### - Hooks

- + Các hàm đặc biệt (bắt đầu bằng use, ví dụ: useState, useEffect) cho phép bạn dùng State và các tính năng khác của React bên trong Functional Component.

- + useState: Quản lý trạng thái nội bộ.

- + useEffect: Xử lý các tác vụ lề (side-effects) như gọi API, lắng nghe sự kiện hoặc tương tác trực tiếp với DOM.

#### - Unidirectional Data Flow (Luồng dữ liệu một chiều)

+ Dữ liệu trong React luôn di chuyển từ trên xuống dưới (từ Component cha xuống Component con thông qua Props), giúp ứng dụng dễ kiểm soát và bảo trì.

### **2.4.3. Ưu điểm và nhược điểm của React.**

- Ưu điểm: Hiệu suất cao (Virtual DOM), dễ tái sử dụng code (Component), hỗ trợ SEO tốt và dễ mở rộng sang ứng dụng di động (React Native).
- Nhược điểm: Chỉ là thư viện UI (cần thêm công cụ ngoài), hệ sinh thái thay đổi quá nhanh và cú pháp JSX gây khó khăn ban đầu cho người mới.

## **2.5. Tổng quan về MongoDB.**

### **2.5.1. Khái niệm về MongoDB**

MongoDB là một database hướng tài liệu (document), một dạng NoSQL database. Vì thế, MongoDB sẽ tránh cấu trúc table-based của relational database để thích ứng với các tài liệu như JSON có một schema rất linh hoạt gọi là BSON. MongoDB sử dụng lưu trữ dữ liệu dưới dạng Document JSON nên mỗi một collection sẽ có các kích cỡ và các document khác nhau. Các dữ liệu được lưu trữ trong document kiểu JSON nên truy vấn sẽ rất nhanh.

MongoDB lần đầu ra đời bởi MongoDB Inc., tại thời điểm đó là thế hệ 10, vào tháng Mười năm 2007, nó là một phần của sản phẩm PaaS (Platform as a Service) tương tự như Windows Azure và Google App Engine. Sau đó nó đã được chuyển thành nguồn mở từ năm 2009.

MongoDB đã trở thành một trong những NoSQL database nổi trội nhất bây giờ, được dùng làm backend cho rất nhiều website như eBay, SourceForge và The New York Times.

- Các feature của MongoDB gồm có: Các ad hoc query, Indexing, Replication, Aggregation, Lưu trữ file

### **2.5.2. Ưu điểm và nhược điểm của MongoDB**

#### **- Ưu điểm**

+ Dữ liệu lưu trữ phi cấu trúc, không có tính ràng buộc, toàn vẹn nên tính sẵn sàng cao, hiệu suất lớn và dễ dàng mở rộng lưu trữ.

+ Dữ liệu được caching (ghi đệm) lên RAM, hạn chế truy cập vào ổ cứng nên tốc độ đọc và ghi cao.

#### **- Nhược điểm:**

---

- + Không ứng dụng được cho các mô hình giao dịch nào có yêu cầu độ chính xác cao do không có ràng buộc.

- + Không có cơ chế transaction (giao dịch) để phục vụ các ứng dụng ngân hàng.

- + Dữ liệu lấy RAM làm trọng tâm hoạt động vì vậy khi hoạt động yêu cầu một bộ nhớ RAM lớn.

- + Mọi thay đổi về dữ liệu mặc định đều chưa được ghi xuống ổ cứng ngay lập tức vì vậy khả năng bị mất dữ liệu từ nguyên nhân mất điện đột xuất là rất cao.

## 2.6. Tổng quan về Docker

### 2.6.1. Khái niệm về Docker

Docker là một nền tảng mã nguồn mở giúp đóng gói, triển khai và vận hành ứng dụng một cách dễ dàng và hiệu quả. Docker sử dụng công nghệ container để cô lập ứng dụng và môi trường của chúng, giúp cho việc triển khai ứng dụng trở nên linh hoạt và nhất quán trên mọi môi trường.

### 2.6.2. Cách thức hoạt động của Docker

Docker sử dụng hình ảnh (image) để đóng gói ứng dụng và môi trường của chúng. Hình ảnh là một tập tin chỉ đọc chứa tất cả các thành phần cần thiết để chạy ứng dụng. Khi bạn tạo một container từ một hình ảnh, docker sẽ tạo ra một môi trường riêng biệt cho ứng dụng chạy.

Sau khi hiểu rõ docker là gì, cách thức hoạt động của docker là gì thì không ít người dùng thắc mắc, liệu thành phần chính của docker gồm những gì. Các thành phần chính của Docker cơ bản gồm:

- Hình ảnh (Image): Tập tin chỉ đọc chứa tất cả các thành phần cần thiết để chạy ứng dụng.
- Container: Môi trường riêng biệt được tạo ra từ một hình ảnh.
- Docker Engine: Phần mềm chạy trên máy chủ để tạo, quản lý và chạy container.
- Docker Registry: Kho lưu trữ các hình ảnh Docker công khai.

### 2.6.3. Ưu điểm và nhược điểm của Docker

| Ưu điểm                                                                     | Nhược điểm                                                        |
|-----------------------------------------------------------------------------|-------------------------------------------------------------------|
| Nhất quán môi trường: Giải quyết triệt để lỗi "chạy được trên máy tôi nhưng | Hiệu năng kém hơn Bare-Metal: Nhỉnh hơn VM nhưng vẫn chịu phí tổn |

|                                                                                                                               |                                                                                                                                         |
|-------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| không chạy được trên máy khách".                                                                                              | (overhead) hiệu năng so với ứng dụng chạy trực tiếp trên hệ điều hành thực.                                                             |
| Gọn nhẹ & Tốc độ: Khởi động trong vài giây, tốn ít tài nguyên hơn hẳn so với Máy ảo (Virtual Machine).                        | Rủi ro bảo mật: Các container chia sẻ chung nhân Kernel của OS host, nếu nhân bị tấn công thì toàn bộ hệ thống có nguy cơ bị ảnh hưởng. |
| Dễ dàng mở rộng & Triển khai: Tương thích tốt với các hệ thống CI/CD và orchestrator như Kubernetes để tự động hóa hạ tầng.   | Không tối ưu cho giao diện (GUI): Thiết kế chủ yếu cho ứng dụng dòng lệnh (CLI) và dịch vụ nền (backend/microservices).                 |
| Quản lý phụ thuộc sạch sẽ: Mỗi container cô lập tài nguyên, không lo xung đột phiên bản phần mềm (node, python, database...). | Quản lý phụ thuộc sạch sẽ: Mỗi container cô lập tài nguyên, không lo xung đột phiên bản phần mềm (node, python, database...).           |

## 2.7. Tổng quan về Render

### 2.7.1. Khái niệm về Render.

Render mang ý nghĩa là kết xuất đồ họa. Nói một cách rõ hơn thì đây có thể xem là quá trình tạo ra một hay nhiều mô hình dưới dạng hình ảnh hai chiều hoặc ba chiều, một nhân vật hoạt hình hay một cảnh phim.

Trong các khâu thiết kế, kết xuất đồ họa là bước quan trọng và đòi hỏi nhiều công sức nhất vì nó thuộc về phần trực quan, tạo ra diện mạo chân thật nhất của mô hình. Kết quả của render được xây dựng bởi cấu trúc dữ liệu của đối tượng cần kết xuất, bao gồm bố cục, điểm nhìn, hình học, chất liệu, cách bố trí ánh sáng,...

Render được sử dụng phổ biến trong:

- Phối cảnh kiến trúc
- Biểu diễn hình tượng của các thiết kế
- Hiệu ứng hoạt hình, phim ảnh,
- Các chương trình mô phỏng,...

### 2.7.2. Render trong các lĩnh vực phổ biến

- Lập trình Web (Web Development):

- Quá trình biên dịch mã HTML, CSS và JavaScript/React thành giao diện hiển thị trên trình duyệt.
  - CSR (Client-Side Rendering): Trình duyệt tự chạy JavaScript để dựng giao diện.
  - SSR (Server-Side Rendering): Máy chủ dựng sẵn HTML rồi mới gửi về cho trình duyệt.
- Đồ họa 3D & Kỹ xảo (3D & VFX): Quá trình máy tính tính toán ánh sáng, bóng đổ, vật liệu (textures) và góc quay từ mô hình 3D để xuất ra hình ảnh hoặc thước phim hoạt hình/điện ảnh cuối cùng (sử dụng phần mềm như Blender, Maya, V-Ray).
- Dựng phim & Sản xuất Video: Quá trình gộp các đoạn video, hiệu ứng, âm thanh, chữ và bộ lọc chuyển cảnh trong phần mềm dựng (Premiere, CapCut) thành một tệp video hoàn chỉnh (như MP4, MOV).
- Game & Xử lý thời gian thực (Real-time Rendering): Máy tính/Card đồ họa (GPU) phải tính toán và dựng hình ảnh cực nhanh (30-144+ khung hình/giây) dựa trên hành vi điều khiển của người chơi

### 2.7.3. Ưu điểm và nhược điểm của Render

- Render.com (Cloud Hosting):
- + Ưu điểm: Dễ dùng (tự động deploy từ Git), hỗ trợ full-stack (Web, API, DB, Docker), SSL/CDN miễn phí, chi phí cố định dễ quản lý.
  - + Nhược điểm: Gói miễn phí bị trễ khởi động (cold start), chi phí cao khi mở rộng lớn, ít tùy biến sâu hệ thống.
- Quá trình Render (Dựng hình Đồ họa / Web):
- + Ưu điểm: Biến code/mô hình 3D thô thành hình ảnh, video và giao diện sống động, chân thực.
  - + Nhược điểm: Tốn nhiều tài nguyên phần cứng (CPU/GPU/RAM) và mất nhiều thời gian xử lý.

## 2.8. Tổng quan về GitHub Actions

GitHub Actions là một nền tảng tích hợp liên tục và phân phối liên tục (CI/CD) cho phép bạn tự động hóa quy trình xây dựng, kiểm thử và triển khai. Bạn có thể tạo các quy trình làm việc để xây dựng và kiểm thử mọi yêu cầu kéo (pull request) vào kho

lưu trữ của mình, hoặc triển khai các yêu cầu kéo đã được hợp nhất lên môi trường sản xuất.

GitHub Actions không chỉ giới hạn ở DevOps mà còn cho phép bạn chạy các quy trình công việc khi các sự kiện khác xảy ra trong kho lưu trữ của bạn. Ví dụ, bạn có thể chạy một quy trình công việc để tự động thêm các nhãn phù hợp mỗi khi ai đó tạo một vấn đề mới trong kho lưu trữ của bạn.

GitHub cung cấp các máy ảo Linux, Windows và macOS để chạy quy trình làm việc của bạn, hoặc bạn có thể tự lưu trữ các máy chủ ảo của riêng mình trong trung tâm dữ liệu hoặc cơ sở hạ tầng đám mây.

## CHƯƠNG 3: HIỆN THỰC HÓA NGHIÊN CỨU

### 3.1 Quy trình nghiên cứu

Đề tài được thực hiện theo quy trình gồm các giai đoạn chính:

- Xác định vấn đề và mục tiêu
- Khảo sát và phân tích nghiệp vụ: Tìm hiểu quy trình hoạt động của thư viện, từ đó xác định các tác nhân và quyền của từng tác nhân.
- Phân tích yêu cầu
- Thiết kế hệ thống
- Xây dựng hệ thống
- Kiểm thử và đánh giá
- Hoàn thiện

### 3.2 Phân tích yêu cầu hệ thống

#### 3.2.1 Yêu cầu chức năng

Hệ thống cần thực hiện các chức năng sau:

*Bảng 1. Yêu cầu chức năng của hệ thống*

| STT | Nhóm chức năng    | Nội dung                             |
|-----|-------------------|--------------------------------------|
| 1   | Quản lý tài khoản | Đăng ký, đăng nhập, xác thực         |
| 2   | Phân quyền        | ADMIN, LIBRARIAN, MEMBER             |
| 3   | Quản lý sách      | Xem, thêm, cập nhật sách             |
| 4   | Quản lý mượn sách | Kiểm tra điều kiện và tạo phiếu mượn |
| 5   | Quản lý trả sách  | Trả sách, cập nhật trạng thái        |
| 6   | Quản lý quá hạn   | Xác định quá hạn và tính tiền phạt   |

#### 3.2.2 Yêu cầu phi chức năng

- Giao diện thân thiện, dễ sử dụng.
- Bảo mật.
- Phân quyền.
- Hiệu năng.

- Tính dễ sử dụng.
- Tính chính xác.
- Tính ổn định.
- Khả năng bảo trì.
- Khả năng mở rộng.
- Tính tương thích.

### 3.3 Cấu trúc use case của hệ thống

*Bảng 2. Danh sách các use case của hệ thống*

| Use Case           | Actor chính              |
|--------------------|--------------------------|
| Đăng ký            | Member                   |
| Đăng nhập          | Admin, Librarian, Member |
| Xem danh sách sách | Admin, Librarian, Member |
| Xem chi tiết sách  | Admin, Librarian, Member |
| Thêm sách          | Admin, Librarian         |
| Cập nhật sách      | Admin, Librarian         |
| Mượn sách          | Member                   |
| Xem phiếu mượn     | Admin, Librarian, Member |
| Trả sách           | Admin, Librarian, Member |
| Xử lý quá hạn      | Admin, Librarian         |
| Tính tiền phạt     | Admin, Librarian         |

### 3.4 Các chức năng chính của hệ thống

#### 1. Quản lý tài khoản và xác thực người dùng

Hệ thống cung cấp chức năng đăng ký và đăng nhập tài khoản. Người dùng có thể tạo tài khoản bằng cách cung cấp các thông tin cần thiết như họ tên, email, mật khẩu và số điện thoại. Khi đăng nhập, hệ thống tiến hành kiểm tra thông tin tài khoản và xác thực người dùng trước khi cho phép truy cập.

Mật khẩu của người dùng được mã hóa trước khi lưu vào cơ sở dữ liệu nhằm đảm bảo an toàn thông tin. Sau khi đăng nhập thành công, hệ thống xác định vai trò của người dùng để cung cấp quyền truy cập tương ứng.

## 2. Phân quyền người dùng

Hệ thống phân chia người dùng thành ba vai trò chính:

**Admin:** quản lý và kiểm soát các chức năng quan trọng của hệ thống.

**Librarian:** thực hiện các nghiệp vụ quản lý sách và quản lý mượn – trả sách.

**Member:** sử dụng các chức năng dành cho thành viên như xem sách, mượn sách, trả sách và theo dõi lịch sử mượn.

Cơ chế phân quyền được thực hiện ở Backend nhằm ngăn người dùng truy cập hoặc thực hiện các chức năng không thuộc quyền hạn của mình.

## 3. Quản lý sách

Chức năng quản lý sách cho phép Admin và Librarian quản lý thông tin sách trong thư viện. Các thông tin được lưu trữ bao gồm tên sách, tác giả, ISBN, thể loại, nhà xuất bản, năm xuất bản, số lượng, số lượng có sẵn, mô tả và trạng thái của sách. Người có quyền quản lý có thể **thêm sách mới và cập nhật thông tin sách**. Các thành viên có thể xem danh sách sách và thông tin chi tiết của từng sách để lựa chọn sách phù hợp.

## 4. Tìm kiếm và xem thông tin sách

Hệ thống cho phép người dùng truy cập danh sách sách và xem thông tin chi tiết của từng đầu sách. Thông tin hiển thị giúp người dùng biết được tên sách, tác giả, thể loại, nhà xuất bản, năm xuất bản và tình trạng số lượng sách có sẵn.

Chức năng này giúp Member có thể kiểm tra thông tin và tình trạng sách trước khi thực hiện thao tác mượn.

## 5. Quản lý mượn sách

Chức năng mượn sách cho phép Member thực hiện yêu cầu mượn một cuốn sách đang có sẵn trong thư viện. Trước khi tạo giao dịch, hệ thống kiểm tra tài khoản người dùng, trạng thái hoạt động của tài khoản, thông tin sách và số lượng sách có sẵn.

Nếu đáp ứng đầy đủ điều kiện, hệ thống tạo phiếu mượn, lưu thông tin người mượn, sách được mượn, ngày mượn và hạn trả. Đồng thời, số lượng sách có sẵn được cập nhật để phản ánh chính xác tình trạng sách trong thư viện.

## 6. Quản lý phiếu mượn

Hệ thống lưu trữ thông tin các giao dịch mượn sách dưới dạng phiếu mượn. Phiếu mượn chứa các thông tin như người mượn, sách được mượn, ngày mượn, hạn trả, ngày trả, trạng thái và tiền phạt nếu có.

Admin và Librarian có thể theo dõi các phiếu mượn theo quyền được cấp, trong khi Member có thể xem các giao dịch mượn sách của bản thân. Chức năng này giúp theo dõi lịch sử và tình trạng các giao dịch trong thư viện.

## 7. Quản lý trả sách

Hệ thống cung cấp chức năng trả sách nhằm hoàn tất một giao dịch mượn. Khi người dùng thực hiện trả sách, hệ thống kiểm tra phiếu mượn, ghi nhận ngày trả và cập nhật trạng thái của phiếu mượn thành đã trả.

Đồng thời, số lượng sách có sẵn được cập nhật tăng lên tương ứng, giúp hệ thống phản ánh chính xác số lượng sách có thể tiếp tục cho mượn.

## 8. Xử lý sách quá hạn và tính tiền phạt

Hệ thống có khả năng xác định trường hợp người dùng trả sách sau thời hạn quy định. Hệ thống so sánh ngày trả thực tế với hạn trả được ghi nhận trong phiếu mượn để xác định số ngày quá hạn.

Nếu phát sinh quá hạn, hệ thống thực hiện tính tiền phạt theo quy định được thiết lập và lưu thông tin tiền phạt vào phiếu mượn. Điều này giúp thư viện dễ dàng theo dõi các trường hợp vi phạm thời hạn trả sách và quản lý khoản phạt tương ứng.

*Bảng 3. Tổng quan các chức năng của hệ thống*

| Nhóm chức năng        | Chức năng chính                                           |
|-----------------------|-----------------------------------------------------------|
| Tài khoản             | Đăng ký, đăng nhập                                        |
| Xác thực & phân quyền | Xác thực người dùng, phân quyền<br>Admin/Librarian/Member |
| Quản lý sách          | Xem, thêm, cập nhật thông tin sách                        |
| Mượn sách             | Kiểm tra điều kiện, tạo phiếu mượn, cập nhật số lượng     |
| Phiếu mượn            | Xem và theo dõi thông tin mượn                            |
| Trả sách              | Trả sách, cập nhật trạng thái và số lượng                 |
| Quá hạn               | Kiểm tra thời gian quá hạn                                |
| Tiền phạt             | Tính và lưu tiền phạt                                     |

### 3.5 Công nghệ sử dụng

Đề tài “Xây dựng ứng dụng quản lý thư viện” được xây dựng dưới dạng ứng dụng web, cụ thể dự án sử dụng các công nghệ sau:

*Bảng 4. Các công nghệ được sử dụng trong đề tài*

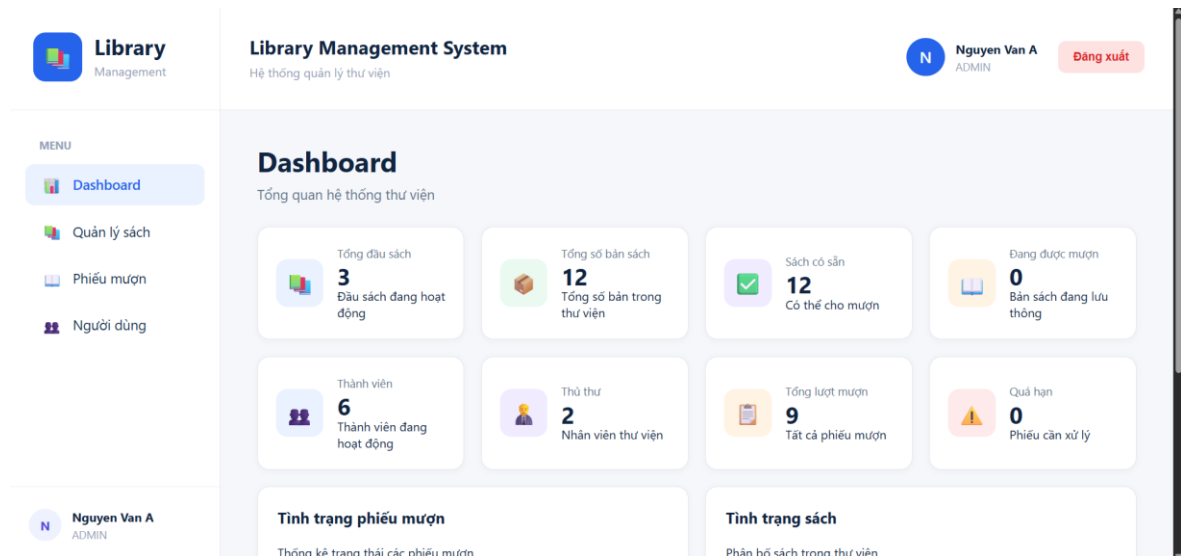
| Thành phần                  | Công nghệ            | Vai trò                                 |
|-----------------------------|----------------------|-----------------------------------------|
| <b>Frontend</b>             | React + Vite         | Xây dựng giao diện người dùng           |
| <b>Backend</b>              | Node.js + Express.js | Xử lý nghiệp vụ và cung cấp RESTful API |
| <b>Database</b>             | MongoDB              | Lưu trữ dữ liệu                         |
| <b>ODM</b>                  | Mongoose             | Kết nối và thao tác với MongoDB         |
| <b>Xác thực</b>             | JWT                  | Xác thực người dùng                     |
| <b>Mã hóa mật khẩu</b>      | bcrypt               | Bảo vệ mật khẩu                         |
| <b>Kiểm thử API</b>         | Postman              | Kiểm tra các API                        |
| <b>Quản lý mã nguồn</b>     | Git + GitHub         | Quản lý phiên bản source code           |
| <b>IDE</b>                  | Visual Studio Code   | Môi trường phát triển                   |
| <b>Đóng gói, triển khai</b> | Docker               | Chuẩn hóa môi trường                    |

## CHƯƠNG 4: KẾT QUẢ NGHIÊN CỨU

Sau quá trình nghiên cứu, tiền xử lý dữ liệu, xây dựng mô hình và phát triển ứng dụng, nhóm đã xây dựng thành công phần mềm quản lý thư viện. Phần mềm được xây dựng dưới dạng ứng dụng web, sử dụng kiến trúc **Frontend – Backend – Database**.

Trong chương này trình bày các kết quả đạt được, giao diện của hệ thống, đánh giá hiệu quả mô hình và trải nghiệm người dùng.

### 4.1 Màn hình tổng quan hệ thống thư viện

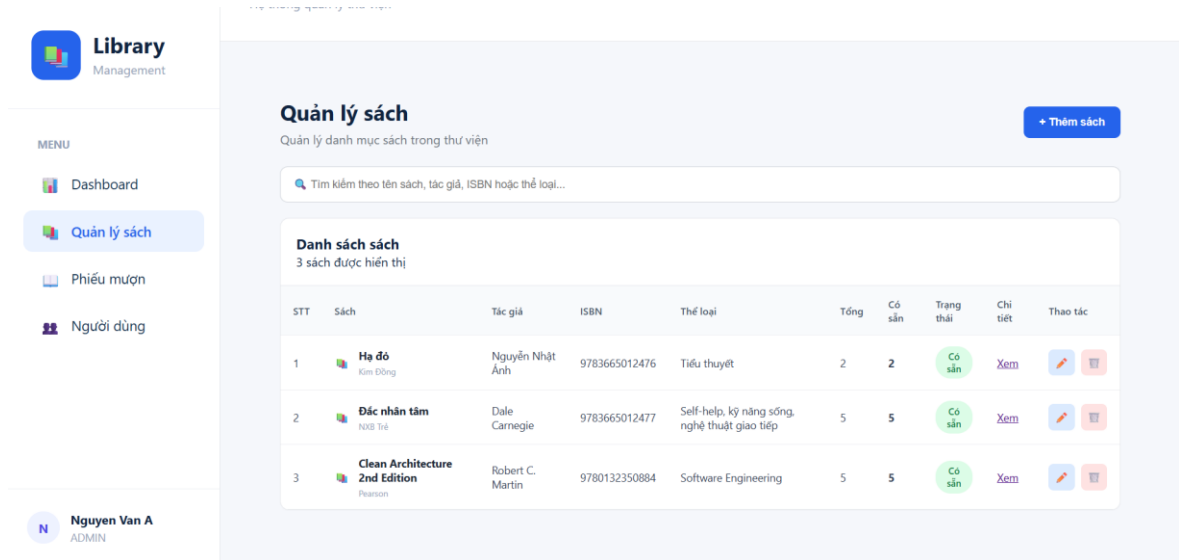


Hình 4.1 Dashboard của hệ thống

Hệ thống đã xây dựng giao diện Dashboard dành cho người quản trị với bố cục rõ ràng, trực quan. Giao diện bao gồm các chức năng chính như:

- Dashboard: cung cấp thông tin tổng quan về tình trạng hoạt động của thư viện.
- Quản lý sách: hỗ trợ quản lý thông tin các đầu sách và số lượng bản sách.
- Phiếu mượn: quản lý quá trình mượn và trả sách.
- Người dùng: quản lý thông tin các thành viên và nhân viên thư viện.
- Đăng nhập và đăng xuất: hỗ trợ xác thực và quản lý phiên làm việc của người dùng.

## 4.2 Quản lý sách



Hình 4.2 Màn hình quản lý sách

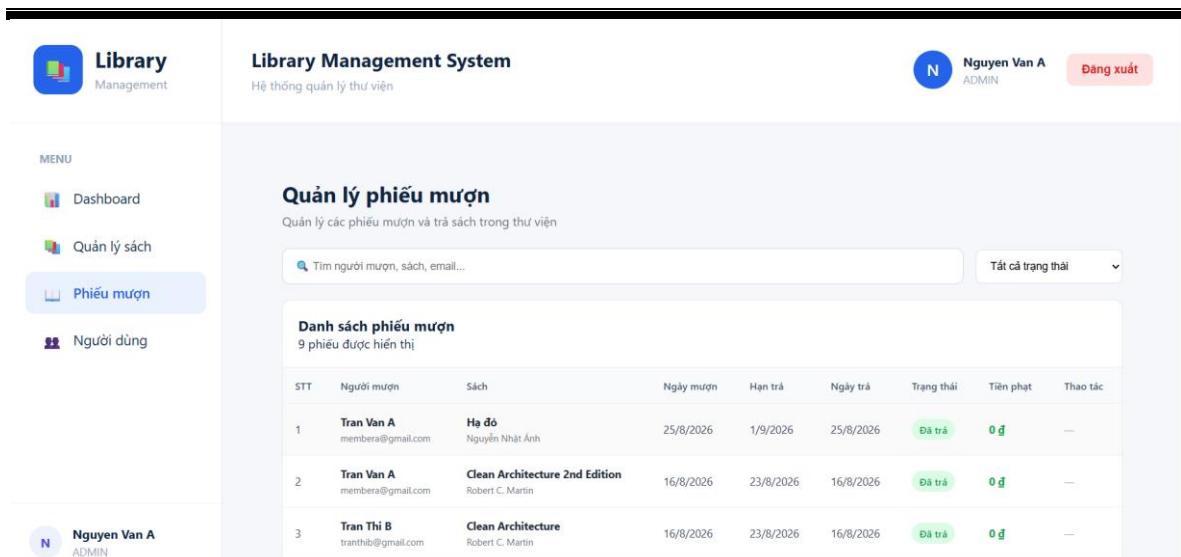
Hệ thống đã hoàn thiện chức năng **quản lý danh mục sách**, cho phép người quản trị theo dõi và quản lý thông tin các đầu sách trong thư viện thông qua giao diện trực quan.

Giao diện quản lý sách cung cấp chức năng **tìm kiếm sách theo tên sách, tác giả, ISBN hoặc thể loại**, giúp người dùng nhanh chóng tìm kiếm và tra cứu thông tin sách. Hệ thống đồng thời hỗ trợ **thêm sách mới, xem chi tiết, cập nhật và xóa sách**, đáp ứng các nghiệp vụ cơ bản trong quá trình quản lý kho sách. Danh sách sách hiển thị đầy đủ các thông tin như **tên sách, tác giả, ISBN, thể loại, tổng số bản sách, số lượng sách hiện có và trạng thái sách**. Trong hình ảnh minh họa, hệ thống đang quản lý 3 đầu sách gồm *Hạ đỏ*, *Đắc nhân tâm* và *Clean Architecture 2nd Edition*, với tổng số lượng lần lượt là 2, 5 và 5 bản sách.

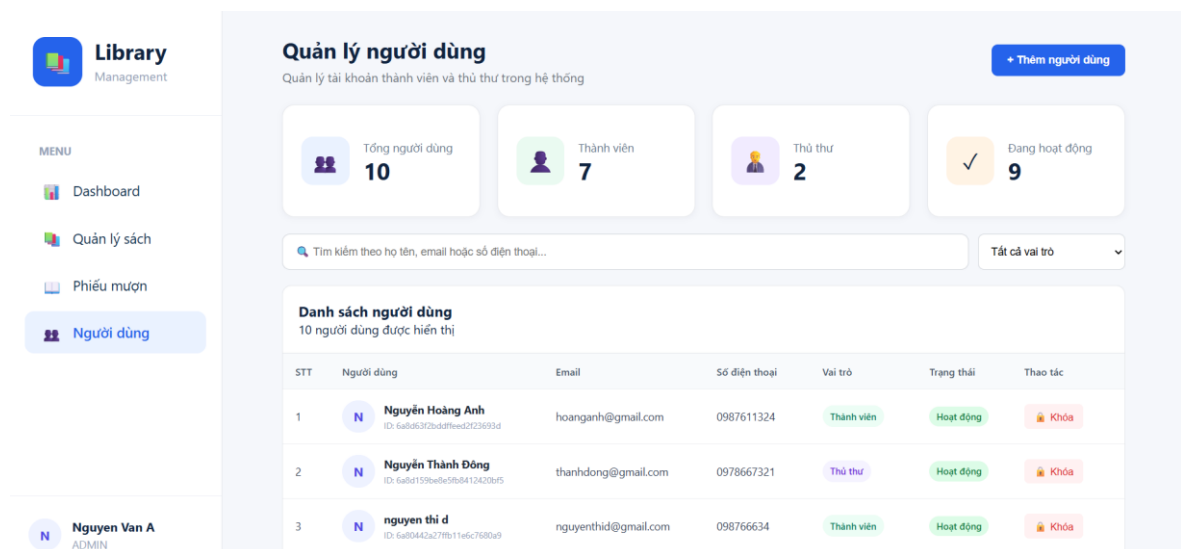
Ngoài ra, hệ thống còn hiển thị trạng thái **“Có sẵn”** đối với các sách đang có thể cho mượn. Các nút thao tác ở cuối mỗi dòng cho phép người quản trị thực hiện các chức năng **xem chi tiết, chỉnh sửa và xóa sách**.

Như vậy, chức năng quản lý sách đã đáp ứng được các yêu cầu cơ bản về **quản lý thông tin sách, tìm kiếm, theo dõi số lượng và cập nhật trạng thái sách**, góp phần hỗ trợ hiệu quả cho quá trình quản lý thư viện.

## 4.3 Kết quả xây dựng mô hình



Hình 4.3 Màn hình quản lý phiếu mượn



Hình 4.4 Màn hình quản lý người dùng

Ngoài ra ứng dụng còn xây dựng được một số chức năng cơ bản phù hợp với hệ thống quản lý thư viện như, quản lý phiếu mượn, phiếu mượn quá hạn, tiền phạt.... và quản lý người dùng, phân quyền người dùng(dành cho admin), tạo người dùng, vô hiệu hóa tài khoản....

## CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

### 5.1 Kết luận

Sau quá trình phân tích, thiết kế và xây dựng, đề tài “**Xây dựng ứng dụng quản lý thư viện**” đã hoàn thành các chức năng cơ bản cần thiết cho một hệ thống quản lý thư viện. Ứng dụng được xây dựng theo mô hình **Frontend – Backend – Database**, sử dụng **React và Vite** cho giao diện người dùng, **Node.js và Express** cho phía máy chủ và **MongoDB** để lưu trữ dữ liệu.

Hệ thống đã triển khai được các chức năng chính như: đăng ký, đăng nhập và xác thực người dùng; phân quyền theo các vai trò **Admin, Librarian và Member**; quản lý sách; quản lý người dùng; thực hiện mượn và trả sách; theo dõi trạng thái mượn sách; tính tiền phạt khi trả sách quá hạn và cho phép thành viên theo dõi lịch sử mượn sách của mình.

Thông qua quá trình kiểm thử, các chức năng chính của hệ thống đã được kiểm tra và hoạt động theo yêu cầu đề ra. Việc phân quyền giúp tăng tính bảo mật và đảm bảo mỗi nhóm người dùng chỉ được phép thực hiện những chức năng phù hợp với vai trò của mình. Hệ thống cũng góp phần giảm bớt việc quản lý thủ công, hỗ trợ nhân viên thư viện theo dõi thông tin sách và hoạt động mượn trả một cách thuận tiện, chính xác hơn.

Bên cạnh những kết quả đạt được, ứng dụng vẫn còn một số hạn chế. Giao diện và trải nghiệm người dùng chưa được tối ưu hoàn toàn, một số chức năng quản lý và báo cáo còn ở mức cơ bản. Hệ thống cũng chưa tích hợp các dịch vụ nâng cao như gửi thông báo tự động, tìm kiếm nâng cao, thống kê trực quan hoặc triển khai trên môi trường production hoàn chỉnh.

Nhìn chung, đề tài đã đáp ứng được mục tiêu xây dựng một ứng dụng quản lý thư viện có tính thực tế, đồng thời giúp áp dụng các kiến thức về phân tích yêu cầu, thiết kế hệ thống, thiết kế cơ sở dữ liệu, lập trình Web, xác thực – phân quyền và kiểm thử phần mềm vào một sản phẩm cụ thể.

## 5.2 Hướng phát triển

Trong tương lai, hệ thống có thể tiếp tục được phát triển theo các hướng sau:

- Hoàn thiện giao diện người dùng: Cải thiện bố cục, khả năng responsive và trải nghiệm sử dụng trên máy tính, máy tính bảng và điện thoại.
- Phát triển chức năng tìm kiếm nâng cao: Cho phép tìm kiếm sách theo tên, tác giả, ISBN, thể loại, nhà xuất bản và nhiều tiêu chí khác.
- Bổ sung chức năng đặt trước sách: Thành viên có thể đặt trước sách đang được mượn và nhận thông báo khi sách có sẵn.
- Xây dựng hệ thống thông báo: Gửi thông báo khi sắp đến hạn trả sách, quá hạn hoặc khi có sách được đặt trước trở lại.
- Phát triển chức năng thống kê và báo cáo: Thống kê số lượng sách, số lượt mượn, sách được mượn nhiều, số người dùng và tiền phạt; đồng thời hiển thị dữ liệu bằng biểu đồ trực quan.
- Bổ sung quản lý hình ảnh và tài liệu: Cho phép quản lý ảnh bìa sách và các thông tin liên quan đến tài liệu.
- Tăng cường bảo mật: Bổ sung JWT nâng cao, refresh token, mã hóa dữ liệu nhạy cảm, giới hạn số lần đăng nhập và các cơ chế bảo vệ API.
- Triển khai hệ thống trên môi trường thực tế: Có thể sử dụng Docker, các nền tảng cloud hoặc máy chủ riêng để triển khai frontend, backend và cơ sở dữ liệu.
- Tự động hóa kiểm thử và triển khai: Xây dựng hệ thống kiểm thử tự động và CI/CD nhằm giúp phát hiện lỗi sớm và hỗ trợ cập nhật phiên bản nhanh chóng.
- Mở rộng quy mô: Trong tương lai có thể phát triển hệ thống thành nền tảng quản lý thư viện trực tuyến cho nhiều chi nhánh, nhiều thư viện hoặc tích hợp với hệ thống quản lý của trường học.

Với những hướng phát triển trên, ứng dụng có thể tiếp tục được hoàn thiện về chức năng, bảo mật, hiệu năng và trải nghiệm người dùng, từ đó đáp ứng tốt hơn nhu cầu quản lý thư viện trong thực tế.

## DANH MỤC TÀI LIỆU THAM KHẢO

- [1] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2015. Tài liệu này phù hợp cho phần quy trình phát triển phần mềm, phân tích yêu cầu, thiết kế hệ thống, kiến trúc và kiểm thử.
- [2] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3rd ed. Boston, MA, USA: Addison-Wesley Professional, 2003. Tài liệu phù hợp cho việc xây dựng Use Case Diagram, Class Diagram, Sequence Diagram và các mô hình UML của hệ thống.
- [3] A. C. Polycarpou, T. Samaras, and J. N. Sahalos, “An RFID-Based Library Management System Using Smart Cabinets: A Pilot Project,” in *Proc. 8th European Conference on Antennas and Propagation (EuCAP)*, The Hague, Netherlands, 2014, pp. 2954–2955, doi: 10.1109/EuCAP.2014.6902447. Đây là tài liệu rất sát với đề tài vì nghiên cứu trực tiếp về hệ thống quản lý thư viện và tự động hóa việc quản lý sách bằng RFID.
- [4] F. Başçıftıcı and L. M. Bokiye, “IoT Based Library Management Automation System Using RFID,” in *Proc. 21st IEEE International Symposium on Computational Intelligence and Informatics (CINTI)*, Budapest, Hungary, 2021, pp. 21–24, doi: 10.1109/CINTI53070.2021.9668584. Tài liệu phù hợp cho phần hướng phát triển hệ thống thư viện thông minh và tự động hóa bằng IoT/RFID.
- [5] MongoDB Inc., “MongoDB Documentation: Data Modeling,” *MongoDB Documentation*, 2026. [Online]. Available: <https://www.mongodb.com/docs/manual/data-modeling/>. Tài liệu được sử dụng để tham khảo cách thiết kế mô hình dữ liệu, quan hệ giữa các document, embedding, referencing và indexing trong MongoDB.
- [6] MongoDB Inc., “MongoDB Documentation: Documents,” *MongoDB Documentation*, 2026. [Online]. Available: <https://www.mongodb.com/docs/manual/core/document/>. Tài liệu mô tả document,

BSON, trường dữ liệu và cơ chế `_id` trong MongoDB, phù hợp với phần thiết kế cơ sở dữ liệu của hệ thống.

[7] MongoDB Inc., “MongoDB Documentation: Development,” *MongoDB Documentation*, 2026. [Online]. Available: <https://www.mongodb.com/docs/development/>. Tài liệu tham khảo cho việc kết nối ứng dụng với MongoDB, thực hiện CRUD, thiết kế dữ liệu và triển khai ứng dụng sử dụng MongoDB.

[8] A. E. Boncătă, R.-M. Nechita, O. Ulerich, and C. Cristoiu, “Transforming Libraries: The Role of Autonomous Robots in Sorting, Inventorying and Book Search,” 2024. Tài liệu phù hợp để tham khảo xu hướng ứng dụng công nghệ tự động hóa trong quản lý, kiểm kê và tìm kiếm sách tại thư viện.

[9] S. He, “A Strategy of Smart Library Construction in the Future,” *Journal of Service Science and Management*, vol. 13, pp. 330–335, 2020. Tài liệu phù hợp với phân tổng quan và hướng phát triển thư viện thông minh, đặc biệt khi mở rộng hệ thống quản lý thư viện truyền thống sang các giải pháp công nghệ hiện đại.

[10] P. D. Devi, S. Mirudhula, and A. Devi, “Advanced Library Management System using IoT,” in *Proc. 2021 Fifth International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud)*, Palladam, India, 2021. Tài liệu phù hợp để tham khảo việc ứng dụng IoT nhằm nâng cao khả năng tự động hóa và quản lý thư viện.